



Build a real-time Data Pipeline using AWS Kinesis and Snowflake

CookBook



Table of Content:

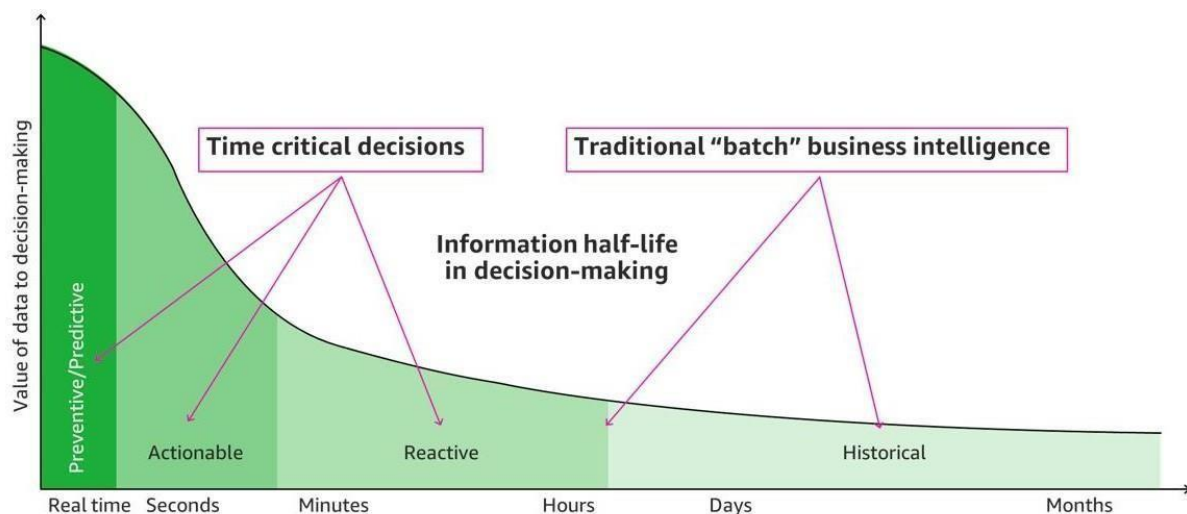
1. Introduction	2
2. Use-case depicted in this project.....	5
i) The Raw-Layer	8
a) Unique Services used in the Raw-Layer.....	8
b) Steps in Raw-Layer	13
c) Code Flow	18
ii) Transformation Layer	21
a) Unique Services used in the Transformation Layer	21
b) Steps in Transformation Layer	27
c) Code Flow	29
d) Code Flow	31
iii) Real-Time layer	40
a) Unique Services used in the Real-Time Layer	40
b) Steps in Real-Time Layer	46
3. Summary	54

Introduction

In this training, we shall look into a pipeline that serves real-time data with the highest value as the data point is generated, corresponding to which an actionable item needs to be initiated. The use cases for the [proposed pipeline](#) can be as follows:

- A global food market system through organizations, vendors and authorities carries a priority level analysis and can be assigned to the end system in real-time.
- A unified system for all agencies, retailers, and departments for handling inquiries and prioritizing the ones generated in real time, irrespective of the influx of data and trends.
- For supply chain management, wherein at the time of emergencies, the organization held in charge can analyze the supply and demand parameters and allocate resources accordingly. The goal is to minimize latency to maximize profits for all included in the ecosystem

The above examples are associated with '[Global Food Market Data](#)' whose value can significantly impart new value over time. When the operating conditions change to the point that the knowledge is helpful. This type of information is time-critical, which means it carries a window for preventive/predictive actions to be taken. On the other hand, Reactive data, which is historical and in 'Batch, real-time generated data' can be used for development in areas such as business intelligence.



Source: Perishable insights, Mike Gualtieri, Forrester

- [Batch vs. Stream Data](#)

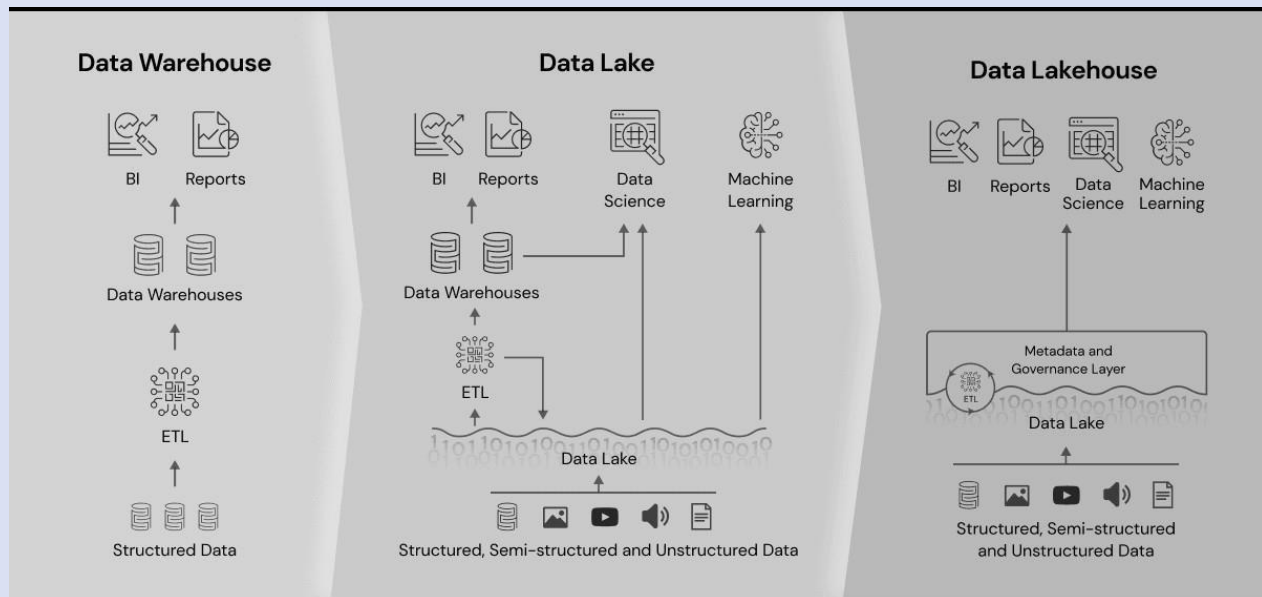
	Batch Processing	Stream Processing
Data Scope	Queries or processing over all or most data in the dataset.	Queries or processing over data within a rolling time window or on the most recent data record.
Data Size	Large batches of data.	Individual records or micro batches consisting of a few records.
Performance	Latencies in minutes to hours.	Requires latency in the order of seconds or milliseconds.
Analysis	Complex Analytics	Simple response functions, aggregates, and rolling metrics.

- Examples of Real-Time data processing
 - Algorithmic Trading: Analyzing real-time market data to execute trades based on predefined algorithms.
 - Patient Monitoring: Collecting and analyzing real-time data from medical devices to monitor patient health continuously.
 - Predictive Maintenance: Monitoring equipment sensors to predict and prevent failures before.
 - Autonomous Vehicles process image data and other sensor data within milliseconds.
- The characteristics of time-critical data can be listed as follows:
 - Staging the data based on the Availability of Data
 - Consistency in storage and Lakehouse solutions
 - Partitioning the data on an evenly distributed metric
 - Effective ingestion of data in the analytical infrastructure of the organization
 - Designing various streams based on the pre-defined actions to be taken
 - Scalability
 - Effective reporting using methods such as visualization
 - Real-time decision and scheduling based on priority

The type of data suitable for this pipeline can originate from an IoT device, server logs, stand-alone applications, or APIs. It can also be delimiter-separated data points generated from live APIs, JSON-like objects, or any other form of raw data generated versus time in small sizes.

Did you know?

To enable business intelligence (BI) and machine learning (ML) on all data, a new, open data management architecture called a “data lakehouse” combines the size, flexibility, and cost-effectiveness of data lakes with the data management and ACID transactions of data warehouses. A novel, open system design that implements comparable data structures and data management capabilities to those in a data warehouse directly on the kind of inexpensive storage used for data lakes makes it possible to create data lakehouses. When combined into one system, data teams can work more quickly since they can use data without visiting numerous systems. Additionally, data lakehouses guarantee that teams working on data science, machine learning, and business analytics initiatives have access to the most complete and current data accessible.



Use-case depicted in this project

[This project](#) will encompass the use case of global food market data and architect a pipeline to help us analyze and take live actions using AWS Kinesis, AWS S3, AWS App Runner, AWS Lambda & Snowflake. Real-time systems are usually architected in such a way on constant patterns but in most cases, they can't scale with peaks in the load resulting in latency and server failures. The services used in this project are highly scalable, robust, agile, on-demand, and cost-effective.

This Project uses the [Global Food Market Data](#), which includes a few of the following fields:

- Country
- Market
- Items
- Prices (divided among other trends)
- High
- Open
- Close
- Indicator
- Trust

For demo purposes, the above data is loaded in a single CSV of 354MB. The data points will be simulated as actual global food markets and generated sequentially in mini-batches through an API.

The pipeline needs to be automated in terms of latency in processing and data availability, as trends such as year and fluctuation of items in different countries/markets can lead to severe drops of significant advt.

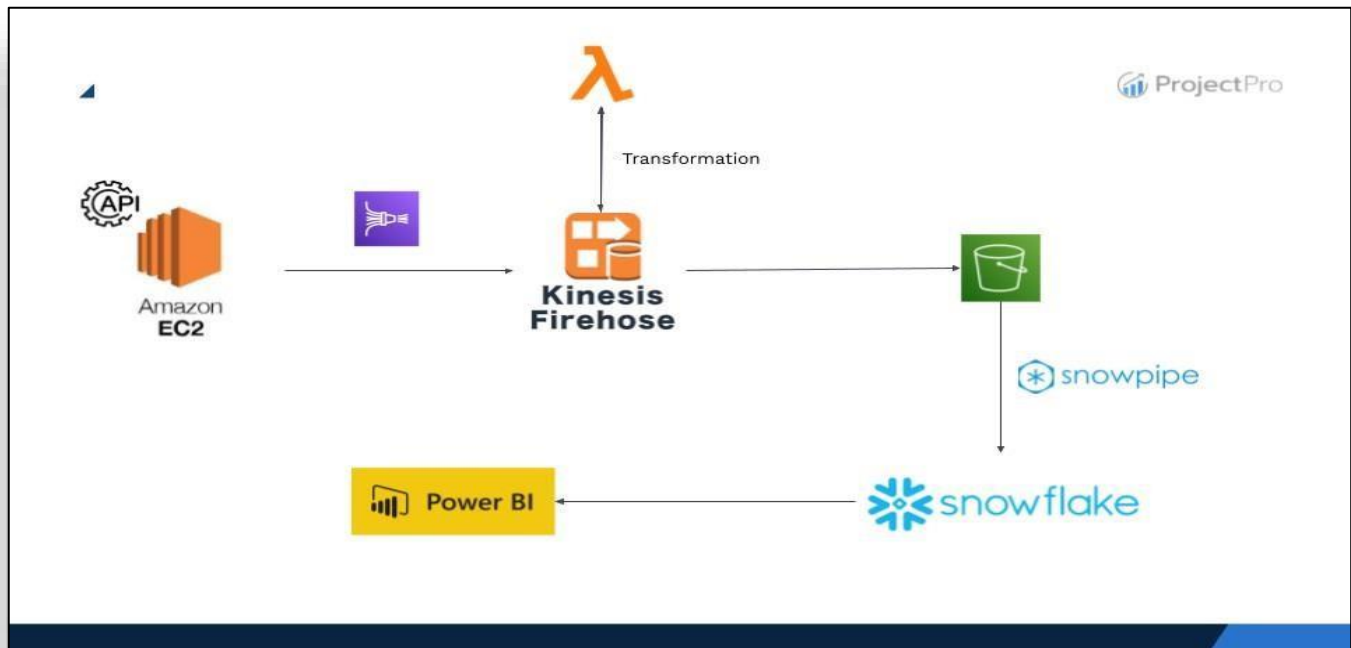
The goal of this project is to have three endpoints:

- A Raw layer- Part of Amazon S3 bucket where the raw data is stored directly and serves as the single source of truth (SSOT) for a Fast API through where data is extracted.
- Analytical layer- This layer stores the filtered and transformed raw data into cleaned/processed data using Snowpark sessions in the Snowflake Warehouse for future reporting, BI applications, and developing data science applications on top of it.
- Real-time layer- The layer serves the cases over AWS Services where data is streamlined through Kinesis Data streams and acts as input for Kinesis Firehose. AWS Kinesis which is a fully managed ETL tool connected with Lambda function to transform real-time logs into CSV form, works as a junction to streamline real-time data from an API. Data Streaming / Incoming Data can be automated through Kinesis Firehose easily with the help of in-house configurations. Notifications are sent quickly to the monitoring system where spikes confirm data streamlining in this layer in different intervals.

Did you Know?

A single source of truth (SSOT) is the practice of aggregating data from many systems within an organization to a single location. An SSOT is not a system, tool, or strategy but a state of being for a company's data in that it can all be found via a single reference point.

The Architecture of the pipeline is as follows:



Breakdown of the Use-Case

- Setup required:
 - [Create an AWS account](#)
 - [Make a GitHub Repository](#)
 - [Download the dataset](#)
 - [Download the Code](#)
 - This is an OS-independent project, as the whole pipeline is set up on AWS Cloud
- Ways to interact with the AWS services:
 - Console- Using the Web-based UI
 - CLI- Using the AWS CLI tool in Terminal/CMD
 - CDK- Using infrastructure-as-a-code, e.g., [CloudFormation](#)
 - SDK- Programmatically, e.g., 'boto3' in python
 - For this project, we will be using the UI Console of AWS to work with different AWS Services and pre-built libraries
- Best Practices for AWS Account
 - [Enable Multi-Factor Authentication\(MFA\)](#)
 - [Protect AWS account](#)
 - Avoid using the root account
 - Choose the AWS region that lies in your location/Country
 - [Create a Budget and set up an alert notification](#)
 - Rotate all keys and passwords periodically
 - Follow the [least privilege principle](#)
 - Follow a naming convention for resources for better understanding, as shown below

Did you Know

```
s3://company-raw-awsregion-awsaccountID-  
env/source/source_region/tablename/year=yyyy/  
month=mm/day=dd/table_<yearmonthday>.<file_format>
```

env = dev, test, prod

source = name or indicator of source

source_region = region of data source

Did you Know?

Give only the rights necessary to complete a task when setting permissions with IAM policies. You achieve this by specifying least-privilege permissions or directly use console change their policies using ARN, commonly known as the activities that can be performed on limited resources under specific circumstances.

The Raw-Layer

Unique Services used in the Raw-Layer

AWS S3

Organizations/ Vendors/ Agencies need help finding, storing, and managing your data. Running applications, delivering high-end supply and demand chains, hosting huge stacks, and backing up emails and other files all necessitated a significant amount of storage. Maintaining the organization's repository was costly and time-consuming for various reasons.

Among the difficulties were the following:

- Purchasing hardware and software components.
- Maintenance necessitates the inclusion of many experts.
- Due to the requirements, there needs to be more scalability.
- Requirements for data security.

Amazon S3 is a cloud storage service that works differently from other popular cloud storage providers. With Amazon S3, data is stored, protected, and retrieved from “buckets” that you can access anywhere, from any device. You store “objects,” such as photos, videos, documents, or other files, in the cloud, and each account can have up to 100 buckets (though you can also request a service limit increase if needed). There's no limit to the number of files you can store in each bucket.

Advantages of AWS S3:

- S3 has a durability of 99.99 percent.
- It offers a variety of “storage classes” for storing data. These classes are based on the frequency and speed with which files must be accessed.
- S3 only charges you for the resources you use, with no hidden fees or overages. You can quickly scale your storage resources to meet the ever-changing needs of your organization.
- In our use case to analyze data thoroughly and simply in real time, S3 is used as a source from which batch of data is called in real time. S3 provides agility among other traditional storage
- It is ideal for many applications, including data storage, backup, software delivery, data archiving, disaster recovery, website hosting, mobile applications, IoT devices, and more.

Disadvantages of AWS S3:

Every cloud storage service has cons, and Amazon S3 is no exception. Here are some of the main disadvantages of Amazon S3:

- Operations on directories are potentially slow and non-atomic.
- Not all file operations are supported. In particular, some file operations needed by Apache HBase are unavailable, so HBase cannot be run on top of Amazon S3.
- Data is visible in the object store once the entire output stream has been written.

- For any “serious” support, they require the “AWS Support Plan,” which is billed separately (around 29 USD per month)
- Downloading data, it’s a bit expensive, 0,09 USD per GB after the first transferred GB/month.
- For beginners using the web interface, it might be “not so intuitive” to set up bucket permissions as it requires configuring additional AWS services like IAM.
- Even if both services don’t have the same set of capabilities, prices are far lower (around 0.005 USD per GB/month)
- It has a complex pricing schema. Even more complicated when we use the Glacier storage class that has both “retrieval time” and “cost per request.”

Alternatives:

- Google Cloud Storage
- Azure Blob Storage
- Wasabi (with no fess for API requests)
- IBM Cloud Object Storage

AWS App Runner

AWS App Runner is a fully managed service that simplifies the deployment of web applications and APIs. It allows developers to deploy applications directly from source code repositories or container images without managing infrastructure. With automatic scaling, App Runner adjusts resources based on traffic demands, ensuring high availability. It offers built-in HTTPS for secure connections and integrates seamlessly with AWS Identity and Access Management (IAM) for security management.

App Runner supports various programming languages and frameworks, making it versatile for different development needs. It automatically handles the build process for source code deployments and provides a secure URL for accessing your application. Additionally, it includes monitoring and logging capabilities through Amazon CloudWatch, facilitating performance tracking and debugging.

Advantages of AWS App Runner:

- **Fully Managed Service:** Eliminates the need to manage servers or infrastructure, allowing developers to focus on building applications.
- **Simplified Deployment:** Easily deploy applications from source code or container images with just a few clicks, streamlining the deployment process.
- **Automatic Scaling:** Automatically scales your application based on traffic, ensuring optimal performance and availability without manual intervention.
- **Built-in HTTPS:** Provides secure HTTPS endpoints by default, ensuring data is transmitted securely without additional configuration.
- **Cost-Effective Pricing:** Pay only for the compute and resources used, making it a flexible and economical choice for various workloads.

Alternatives:

- Google Cloud Run
- Azure App Service
- Heroku
- Render
- Digital Ocean App Platform
- Vercel
- Netlify

Amazon Kinesis

Real-time streaming data can be easily collected, processed, and analyzed with Amazon Kinesis to gain immediate insights and respond rapidly to new information. With the freedom to select the tools that best meet your application's needs, Amazon Kinesis provides essential features for processing streaming data at any scale economically. You may ingest real-time data for machine learning, analytics, and other applications with Amazon Kinesis, including video, audio, application logs, website clickstreams, and IoT telemetry data. Instead of waiting until all your data has been collected before processing, Amazon Kinesis lets you process and analyze data as it comes in and responds immediately.

Services under Kinesis:

- **Kinesis Data Streams**
 - Collect and Store Data Streams for analysis
 - Streams stores data in 'Shards' with each shard serving 1MB/s OR 1000 records/s. The field selected for shards(partitions) should be balanced.
 - The data retention period is 24 hours by default, which can be increased for up to 7 days and further increased upon request to AWS.
 - When the data stream throughput is unpredictable and auto-scaling is required, one can use the 'On-Demand' configuration. This configuration provides a writing capacity of a maximum of 200 MB/s OR 200,000 records/s and a read capacity of 400 MB/s
 - If the throughput increases more than 200 MB/s or better control is needed over the number of shards created, the 'Provision' configuration can be selected.
- **Kinesis Data Firehose**
 - Amazon Kinesis Data Firehose is a fully managed service to deliver real-time streaming data to destinations such as Amazon S3 and Redshift.
 - You don't need to write applications or manage resources with Kinesis Data Firehose. You configure your data producers to send data to Kinesis Data Firehose, and it automatically delivers the data to the destination that you specified. You can also configure Kinesis Data Firehose to transform your data before returning it.
 - The data of interest that your data producer sends to a Kinesis Data Firehose delivery stream. A record can be as large as 1,000 KB.
 - No facility for data storage.
 - Near real-time processing capabilities, depending on the buffer size or minimum buffer time of 60 seconds.

- Everything needed to run your real-time applications constantly is handled by Amazon Kinesis Data Analytics, which scales automatically to match the volume and throughput of your incoming data. You only pay for the resources your streaming applications use using Amazon Kinesis Data Analytics. There are no setup fees or minimum charges.

Advantages of Amazon Kinesis:

- Most analytics tools either delay processing, sample data across the stream, or process streaming data in chunks. Amazon Kinesis Data Analytics performs processing on the data stream in real time.
- You can gain real-time insights from machine learning on streaming data. Your time to insights will be almost immediate because of Kinesis Data Analytics' processing power.
- Because Amazon Kinesis Data Analytics is serverless, you don't have to bother setting up and managing servers; the underlying infrastructure is handled and automated in the background, allowing you to concentrate on your data analytics rather than server maintenance.
- In most cases, Kinesis Data Analytics elastically adjusts your application to consider your source stream's data throughput and query complexity. By setting up the parallel execution of tasks and resource allocation, scaling can be enabled.

Assume you bundle your incoming data streams using AWS Kinesis Data Analytics and Amazon Kinesis Data Streams. Then, it conveniently enables you to perform standard processing across the board, and with this setup, there is no data transmission price. The records for Data Streams are chunked into 25KB units, and each shard can handle 1MB/sec or 1000 PUT records per second. Shards are made accessible by the hour, and up to 2 MB/s can be produced. Stream data are typically accessible for up to 24 hours.

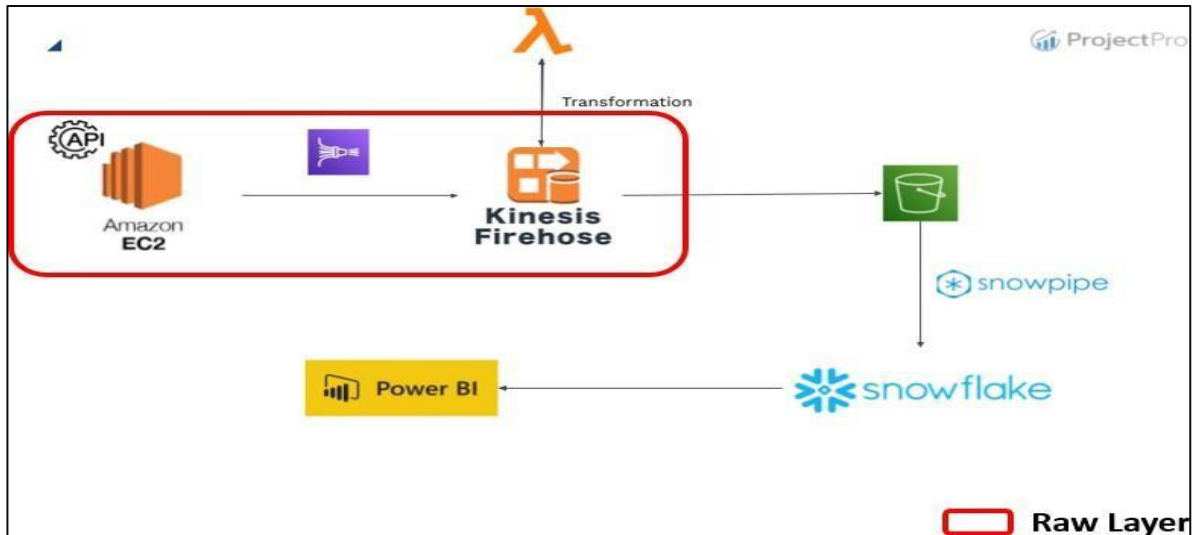
- The Amazon Kinesis Data Analytics service uses a pay-as-you-go business model, like many other AWS services. As a result, no wasted resources or tasks are required to optimize the underlying resources; you only pay for what you utilize (beyond the streaming data you throughput).
- No need to master brand-new programming languages, frameworks, or machine learning techniques.

With Amazon Kinesis Data Analytics, the learning curve for new products is considerably shortened because you don't need to learn any new languages or frameworks to get started. You may quickly put together your preferred data analytics operations with the help of the service, which contains machine learning algorithms and preconfigured SQL procedures.

Alternatives:

- Google Cloud Dataflow.
- Google Cloud Pub/Sub.
- Spark Streaming.
- Apache Flink.
- Azure Event Hubs.
- Azure Stream Analytics.
- Apache Kafka

Raw-Layer



Steps in Raw-Layer:

1. [Create an S3 bucket](#)
2. Upload RAW data file in that S3 bucket
3. Change the accessibility of the S3 Bucket to the public using policies
 - a) Go to the Permissions Tab of your S3 bucket.
 - b) Select Edit Policy
 - c) Write a JSON Policy to change the access of your s3 bucket to public.

Policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": "*",
7       "Action": "s3:GetObject",
8       "Resource": "arn:aws:s3:::bucket-name/*"
9     }
10  ]
11 }
  
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

- d) Replace "bucket-name" with your S3 Bucket Name.

4. [Setup GitHub Repository](#)

5. Create Two files in GitHub Repository
 - a) Create a Python file (.py) containing FastAPI code to build a FastAPI application.

Code Flow :

```
#Import Necessary Libraries for FastAPI Application

# Import FastAPI and Query for creating the API and handling query

from fastapi import FastAPI, Query
import pandas as pd # Import pandas for data manipulation
import boto3 # Import boto3 for AWS S3 interactions (if needed)
from io import StringIO # Import StringIO handling string input/output
import uvicorn # Import uvicorn for running the FastAPI app

# Create an instance of FastAPI

# FastAPI application for fetching and filtering data from a CSV file in S3

app = FastAPI()
def fetch_data(year: int = None, country: str = None, market: str = None):

    """
    Fetch and filter data from a CSV file hosted on S3 based on provided
    parameters.

    Parameters:
    - year: Filter for a specific year (optional).
    - country: Filter for a specific country (optional).
    - market: Filter for a specific market (optional).

    Returns:
    - JSON string of filtered records or an error message.
    """

    try:
        # Load CSV content into a pandas DataFrame from the S3 URL

        df = pd.read_csv("https://ap-south-1/total_data.csv")

        print(df.shape[0]) # Print the number of rows in the original DataFrame

        # Apply filters based on provided parameters

        if year is not None:
            print("Filtering by year")
            df = df[df['year'] == year]
```

```
if country is not None:
    print("Filtering by country")
    df = df[df['country'] == country]
if market is not None:
    print("Filtering by market")
    df = df[df['mkt_name'] == market]

# Fill NaN values with empty strings for cleaner output
df_filter = df.fillna('')

print(df_filter.shape[0]) # Print the number of rows after filtering

# Convert filtered DataFrame to JSON
if df_filter.empty:
    raise ValueError('No data found for the specified filters.')
# Raise error if no data found
else:
    filtered_json = df_filter.to_json(orient='records')
    return filtered_json

except Exception as e:
    return {'error': str(e)}

@app.get('/fetch_data') # Define an endpoint for the API
async def fetch_data_api(year: int = Query(None), country: str = Query(None),
    market: str = Query(None)):
    try:
        # Call fetch_data function with provided parameters
        filtered_data = fetch_data(year, country, market)

        if filtered_data is None:
            raise ValueError('No data found for the specified filters.')
        else:
            # Return filtered data as API response
            return filtered_data
    except Exception as e:
        return {'error': str(e)}, 400

if __name__ == '__main__':
    # Run the FastAPI application using uvicorn server
    uvicorn.run(app, port=8080, host='0.0.0.0') # Specify the host and port for
    the server
```

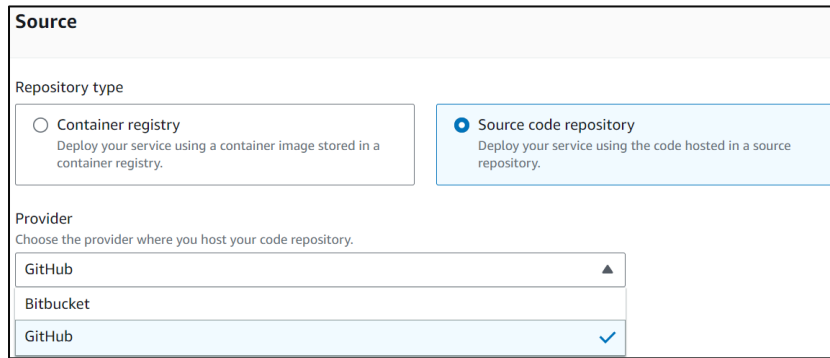
- b) Create a (.txt) text File in the repository for Required Libraries.

```

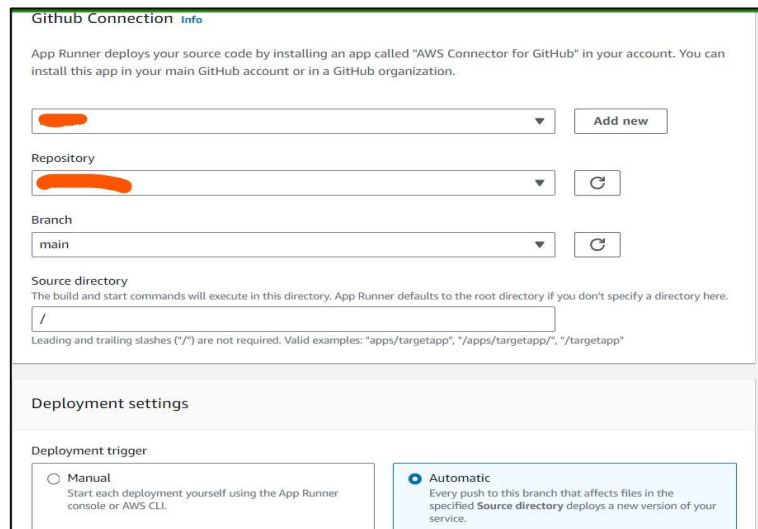
Numpy          # Library for numerical computing for arrays and matrices.
Pandas         # Library used for data analysis and processing CSV files.
Requests==2.31.0 # Library for making requests in Python, for API interactions.
urllib3==1.26.6 # user-friendly HTTP client for Python often used with Requests
fastapi==0.85.1 # web framework for building APIs with automatic interactive.
mangum==0.16.0  # An adapter for running ASGI applications on AWS Lambda
pydantic==1.10.2 # A data validation and settings management library
uvicorn==0.19.0  # An ASGI server for running asynchronous web applications,
starlette==0.20.4 # The lightweight ASGI framework that FastAPI is built upon
boto3          # The AWS SDK for Python for interaction with AWS services like S3.

```

6. Deploy an API with AWS App Runner Using FastAPI Source Code from GitHub Repository
- Navigate to the AWS App Runner service in your AWS account.
 - Choose "GitHub" as the source for your application deployment.



- Authorize AWS App Runner to access your GitHub account and select the appropriate repository.
- Specify build settings, including the branch to deploy and any necessary build commands.



- e. Specify build settings, specify port, start Command and any necessary build commands.
- f. Specify your GitHub Repo file name in Build and Start Command.

Build settings

Configuration file

☒ **Configure all settings here**
Specify all settings for your service here in the App Runner console.

☐ **Use a configuration file**
Let App Runner read your configuration from the `apprunner.yml` file in the source directory of your code repository. App Runner defaults to the root directory if a **Source directory** wasn't specified in the previous step.

Runtime

Choose an App Runner runtime for your service.

Python 3

Build command

This command runs in the source directory of your repository when a new code version is deployed. Use it to install dependencies or compile your code. App Runner defaults to the root directory if a **Source directory** wasn't specified in the previous step.

pip install -r requirements.txt

Start command

This command runs in the source directory of your service to start the service processes. Use this command to start a webserver for your service. The command can access environment variables that App Runner and you defined. App Runner defaults to the root directory if a **Source directory** wasn't specified in the previous step.

python app.py

Port

Your service uses this TCP port.

8080

Cancel

Previous

Next

- g. Review your configurations and click "Create and deploy" to launch your FastAPI API.
- h. After deployment, use the provided domain to access your API endpoints.

Service overview

Status

Running

Incoming traffic

Public endpoint

Default domain

[https://github.com/...](#)

Service ARN

arn:aws:apprunner:...

Source

<https://github.com/>

7. After implementing this steps, Create Kinesis Data Streams using AWS KINESIS
 - a. Create Data Streams from Kinesis Dashboard
 - b. Create a unique name for your data stream in the "Data stream name" field.
 - c. Once created, your new data stream will appear in the list, and you can start sending data to it.
8. Create Code for Streaming Data from API to Kinesis Data Streams
 - a. Create a python file (.py) and import all necessary libraries and statements
 - b. Initialize the Kinesis Client to Stream Real-Time Data from an API to Kinesis Data Stream
 - **Kinesis Client** : The **Kinesis Client** is a library provided by AWS SDKs, like Boto3 for Python, that enables applications to interact with Amazon Kinesis Data Streams. By using the Kinesis Client, you can efficiently put records into streams for further processing and analysis.

Code Flow :

Step 1 : Setting up Kinesis Client

```
#Initialize Kinesis Client
import boto3
kinesis_client = boto3.client('kinesis', region_name='ap-south-1', aws_access_key_id='',
                               aws_secret_access_key='')

```

Step 2 : Define send_to_kinesis Function (takes data as an input)

- send_to_kinesis(data) takes data as input, converts it into a JSON string (data_record) using json.dumps().
- kinesis_client.put_record() sends data_record to an Amazon Kinesis stream named 'akash-kinesis-datastream'.

```
#Define Kinesis Streams
import requests
import boto3
import json
import time
import pandas as pd

# Initialize Kinesis client

def send_to_kinesis(data):
    # Send data to Kinesis stream
    data_record = json.dumps(data)
    kinesis_client.put_record(StreamName='akash-kinesis-datastream', Data=data_record, PartitionKey='1')

```

Step 3: Calling an API and send data to Kinesis('invoke_api' function)

```
#Calling an API using requests
def invoke_api():
    # API Gateway endpoint URL

    api_endpoint = ''
    # Define the parameters
    params = {
        'year': ['2009', '2010', '2012', '2013'],
        'market': 'Armavir'
    }

    # Make a GET request to the API
    response = requests.get(api_endpoint, params=params)

    json_data = json.loads(json.loads(response.text))

    print(len(json_data))

```

Step 4: Sending Data to Kinesis

- print(len(json_data)) prints the number of items received from the API response.
- The for loop iterates through json_data and calls send_to_kinesis(item) to send each item to the Kinesis stream.

```
response = requests.get(api_endpoint, params=params)
json_data = json.loads(json.loads(response.text))
print(len(json_data))

for i in range(0, len(json_data)):
    send_to_kinesis(json_data[i])

# print(i)

invoke_api()

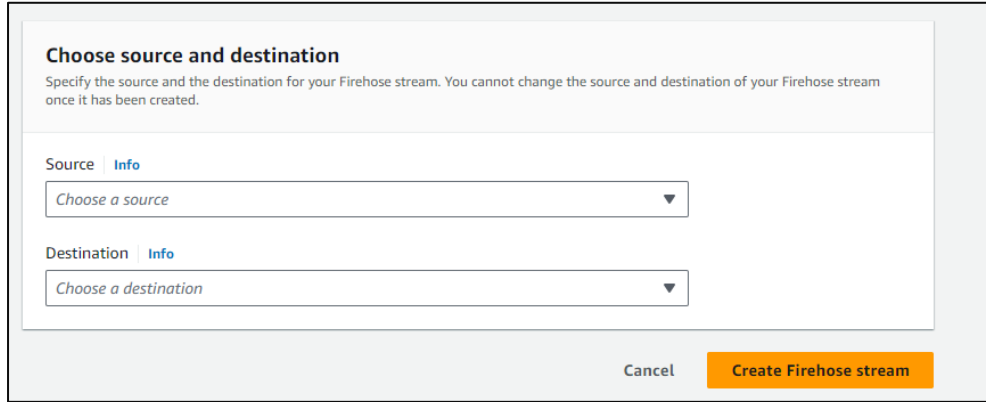
```

Output

```
[{"country": "Armenia", "mkt_name": "Armavir", "DATES": "11/1/2008",
  "inflation_apples": 1236.39, "inflation_apples": 1236.39, "trust_apples": 8.1, "o_bananas": 1.0,
  "o_beans": 1.0, "h_beans": 1.0, "l_beans": 1.0, "c_beans": 1.0,
  "c_bread": 391.87, "inflation_bread": 391.87, "trust_bread": 6.7, "o_bulgur": 1.0,
  "o_cabbage": 143.31, "h_cabbage": 161.63, "l_cabbage": 130.61, "c_cabbage": 1.0}

```

9. Create Kinesis Data Firehose using AWS data Firehose service.
 - a. Select your Kinesis Data stream as your source from the drop-down list .
 - b. Create another S3 bucket as your destination point for real-time streaming data.
 - c. Select your newly created S3 Bucket as a destination.



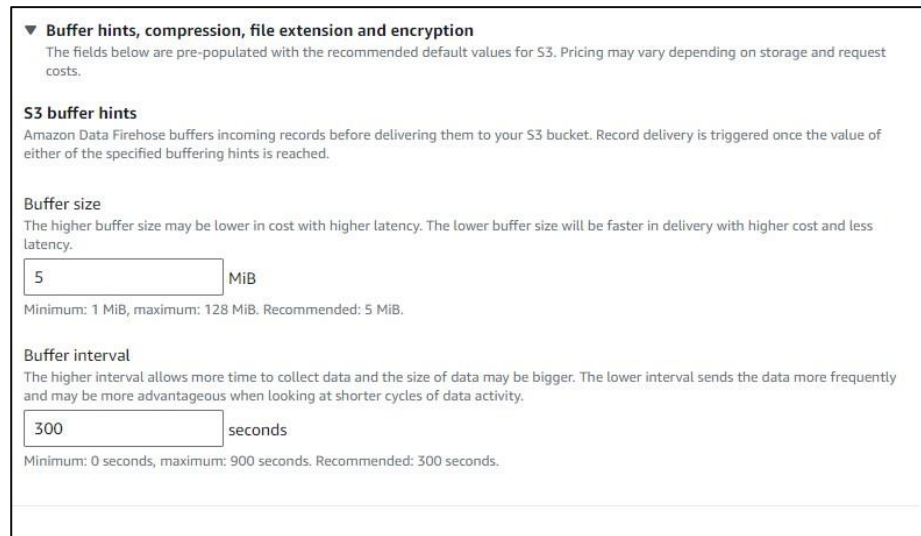
Choose source and destination
Specify the source and the destination for your Firehose stream. You cannot change the source and destination of your Firehose stream once it has been created.

Source [Info](#)

Destination [Info](#)

[Cancel](#) [Create Firehose stream](#)

- d. Configure the buffering settings, such as buffer size and buffer interval, to control how data is batched before being sent to S3.



▼ **Buffer hints, compression, file extension and encryption**
The fields below are pre-populated with the recommended default values for S3. Pricing may vary depending on storage and request costs.

S3 buffer hints
Amazon Data Firehose buffers incoming records before delivering them to your S3 bucket. Record delivery is triggered once the value of either of the specified buffering hints is reached.

Buffer size
The higher buffer size may be lower in cost with higher latency. The lower buffer size will be faster in delivery with higher cost and less latency.

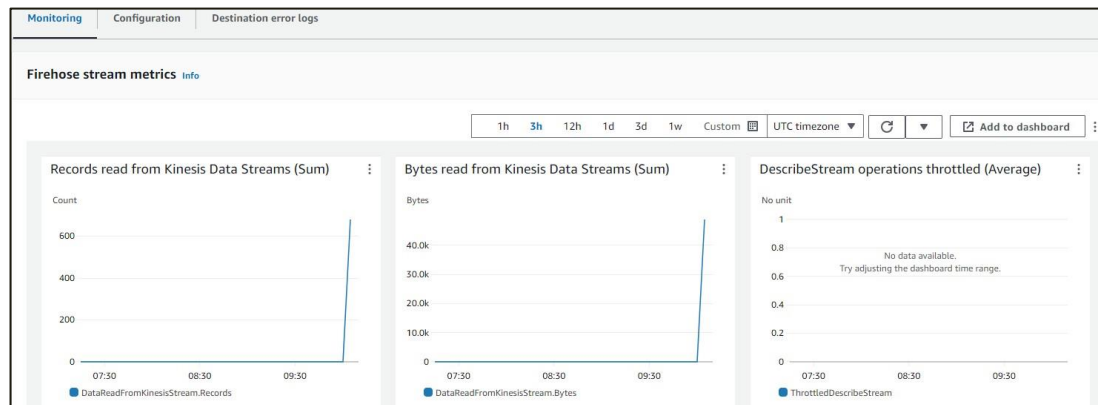
MiB
Minimum: 1 MiB, maximum: 128 MiB. Recommended: 5 MiB.

Buffer interval
The higher interval allows more time to collect data and the size of data may be bigger. The lower interval sends the data more frequently and may be more advantageous when looking at shorter cycles of data activity.

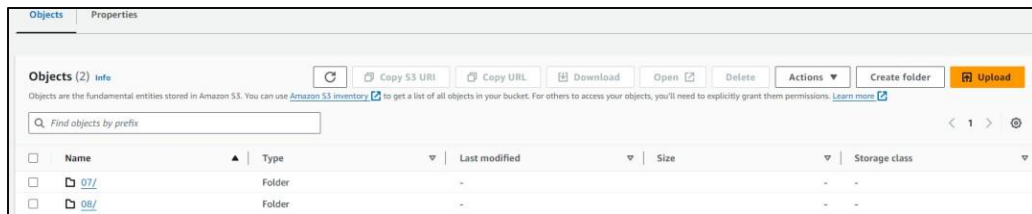
seconds
Minimum: 0 seconds, maximum: 900 seconds. Recommended: 300 seconds.

- e. Once the Kinesis Firehose is created, you can monitor it by invoking an API that sends data to Kinesis Data Streams. The data through these streams will then be available in the Kinesis Firehose.

- f. Check the monitoring in AWS Kinesis Firehose to see if you can observe the spikes which confirm the real-time data ingestion.



- g. Check that the data is stored in the destination S3 bucket in partitioned format, which facilitates efficient querying and retrieval.

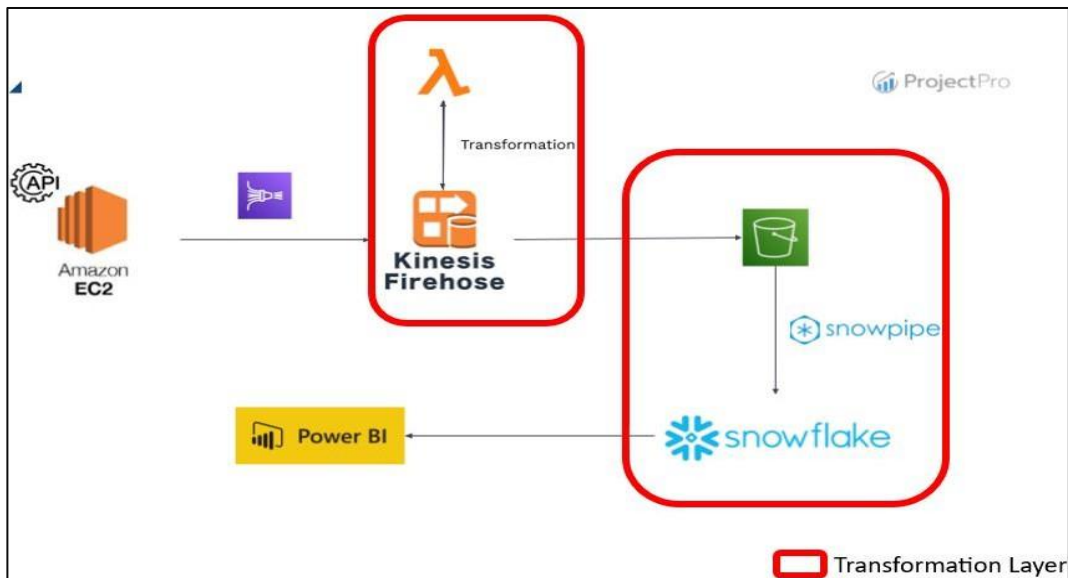


The screenshot shows the 'Objects' tab of the AWS S3 console. It displays a list of objects in a bucket, including folders '07/' and '08/'. The table has columns for Name, Type, Last modified, Size, and Storage class. The '07/' folder is highlighted.

Name	Type	Last modified	Size	Storage class
07/	Folder	-	-	-
08/	Folder	-	-	-

Transformation layer

Unique Services used in the Transformation Layer



Snowflake:

What sets Snowflake apart is its comprehensive support for various data types, including semi-structured and unstructured data, positioning it ideally for advanced applications such as machine learning and AI. Acting as both a data warehouse and a data lake, Snowflake incorporates external stages for preprocessing data before it reaches the main storage layer. This capability streamlines real-time and batch data handling, offering a versatile solution adaptable to dynamic business needs. Its innovative three-tier architecture distinguishes it by separating storage from compute, which not only enhances scalability and concurrency but also supports simultaneous querying and analysis by multiple users.

In this project, Snowflake and its capabilities were leveraged to manage data effectively from an S3 bucket through automated pipelines. Snowpipe, an automated data ingestion feature, was employed to seamlessly load raw data from an external S3 stage into Snowflake's raw tables. This automated process ensured timely and efficient data ingestion without manual intervention.

Subsequently, Snowpark sessions were utilized to process large datasets efficiently. This involved tasks such as filtering out unnecessary columns, optimizing their placement for better query performance, and managing upsert operations based on row counts.

Alternatives:

- Big Query
- AWS Redshift

Did you know?

The Top 3 Apache Projects are:

- Camel
- Flink
- Apache Airflow

Did you Know?

Stateful Functions is an API that simplifies building distributed stateful applications. It's based on functions with a persistent state that can interact dynamically with strong consistency guarantees.

Snowflake Snowpipe

Organizations can easily extract their data automatically from raw layer to snowflake warehouse using snow pipe which cut down the manual effort of loading every single updated data from raw layer. Snowflake Snowpipe is a feature designed for automated data ingestion into Snowflake from external data sources, such as Amazon S3. It operates based on the "continuous ingest" model, where new data arriving in the external stage triggers the automatic loading process into Snowflake tables. This real-time or near-real-time data ingestion capability eliminates the need for manual intervention and ensures that the data in Snowflake remains up-to-date with changes happening in the external data sources.

Snowpark Sessions

Effective Snowpark is a feature of Snowflake that allows users to write and execute Spark-compatible code directly within Snowflake. It integrates the power of Spark's processing capabilities with Snowflake's data management and querying abilities, providing a unified platform for data engineering and analytics tasks.

Key Features of Snowpark:

- **Integration with Snowflake:** Snowpark allows seamless integration with Snowflake's data ecosystem, enabling users to leverage Snowflake's data warehouses and data lakes for data processing.
- **Support for Spark APIs:** Snowpark provides APIs that are compatible with Apache Spark, including DataFrame and SQL APIs. This compatibility allows users familiar with Spark to work efficiently within Snowflake.

In a specific project scenario:

- **Objective:** The project involved processing raw data from a stage layer in Snowflake using Snowpark sessions written in Python scripts.
- **Data Cleansing and Transformation:** Snowpark sessions were used to clean and transform raw data, including filtering out unnecessary columns, standardizing column positions for better alignment with analytics needs, and ensuring data quality by handling upserts based on row counts.
- **Efficiency Measures:** To optimize compute costs, row count techniques were implemented to selectively process new data since the last execution, thereby reducing unnecessary scans of the entire dataset.
- **Data Loading and Visualization:** The cleaned data was then loaded into designated Snowflake tables, serving as a reliable data source for generating visualizations and conducting further analysis.

Advantages of Snowpark:

- **Unified Platform:** Snowpark provides a unified environment where data engineers and analysts can perform ETL (Extract, Transform, Load) tasks, data cleansing, and advanced analytics without moving data across different platforms.

Disadvantages of Snowpark:

- **Learning Curve:** Users unfamiliar with Spark may face a learning curve when adapting to Spark-compatible APIs and Snowflake's data management concepts.
- **Platform Limitations:** While powerful, Snowpark may have certain limitations compared to full-fledged Spark clusters in terms of customizability and specialized use cases.

Snowflake Stored Procedures:

Snowflake stored procedures are SQL scripts stored within Snowflake that can be executed as a single unit of work. They are used to encapsulate complex SQL logic or operations, enabling users to automate tasks, enforce business rules, and streamline data processing workflows within Snowflake.

Key Features:

SQL-Based: Stored procedures in Snowflake are written in SQL, allowing for familiar syntax and integration with Snowflake's data handling capabilities.

Modularity: Procedures can contain multiple SQL statements and control logic (loops, conditionals), making them suitable for implementing complex data transformations and business logic.

Transaction Management: Transactions within stored procedures ensure that operations either complete successfully as a whole or are rolled back to maintain data consistency.

Snowflake Tasks:

Snowflake tasks are scheduling objects used to automate the execution of stored procedures or SQL statements at specified intervals or times. They are designed to automate routine tasks, data integration processes, and maintenance activities within Snowflake.

Key Features:

Automation: Tasks automate the execution of predefined actions, such as running stored procedures or SQL statements, at scheduled times or intervals.

Flexibility: Tasks can be configured to run one-time or recurrently (daily, weekly, monthly) based on specified schedules or triggers.

In this project, Snowflake stored procedures and tasks were utilized to automate the data cleaning and transformation processes using Snowpark sessions. The objective was to ensure that raw data from an S3 bucket was consistently processed and prepared for analytics without manual intervention.

- **Implementation:**
 - **Snowpark Sessions:** Python-based Snowpark sessions were developed to cleanse and transform raw data from the raw/stage layer in Snowflake. These sessions standardized column positions, handled upserts, and ensured data quality by filtering out irrelevant columns and rows with missing values.
 - **Stored Procedures:** Snowflake stored procedures encapsulated the Snowpark sessions, defining the sequence of operations and logic required to execute the data cleaning tasks automatically.
 - **Task Creation:** Snowflake tasks were created in the public schema of the database to schedule the execution of stored procedures (containing Snowpark sessions) at specified intervals. This automation ensured that new data from the S3 source was processed promptly upon arrival, maintaining data freshness and reliability for analytics.

Advantages and Disadvantages of Stored Procedures:

Advantages:

- **Automation:** Stored procedures automate complex data processing tasks, reducing the need for manual intervention and improving operational efficiency.
- **Consistency:** Procedures enforce standardized data transformations and business rules, ensuring consistent data quality across processes.
- **Transaction Control:** They provide transactional integrity, allowing atomic operations that either fully succeed or roll back in case of errors.
- **Reuse and Modularity:** Procedures can be parameterized and reused across different datasets or scenarios, promoting code reusability and maintainability.

Disadvantages:

- **Complexity:** Developing and maintaining complex stored procedures may require specialized SQL skills and careful design to avoid performance bottlenecks or errors.
- **Vendor Lock-in:** Procedures tied to Snowflake may limit portability to other platforms or databases, affecting system flexibility in multi-cloud environments

Alternatives:

- Google BigQuery
- Amazon Redshift

AWS Lambda:

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the computing time you consume—there is no charge when your code is not running. With Lambda, you can run code for virtually any application or backend service—all with zero administration. Just upload your code, and Lambda takes care of everything required to run and scale your code with high availability. You can set up your code to trigger automatically from other AWS services or call it directly from any web or mobile app.

Advantages of AWS Lambda:

Many of the advantages of AWS Lambda relate to the advantages of adopting a serverless approach.

- **Saves Time and Effort:** One of the significant benefits of going serverless using AWS Lambda is the time and effort saved from creating and maintaining your application's infrastructure.
- AWS saves a lot of effort by provisioning and managing the infrastructure your Lambda functions run on.
- It automatically scales the instances to handle excessive load times and implements a proper logging and error-handling system. Those involved in constructing or maintaining infrastructure will reasonably understand the gravity of this advantage.
- Not only does it save you time while building the application, but it will also save you ample time required to maintain an already established system as your application evolves and scales up.
- Lambda ensures quicker production and greater agility in launching your product in the market, thus giving you an edge over your competition.
- **Helps in building a scalable solution:** AWS Lambda has become one of the most popular solutions in such a short span of time because of its ability to accommodate applications at scale and early stages.
- For large loads of applications, AWS runs your Lambda function simultaneously with other Lambda functions. This means that you do not have to worry about clogged-up queues.
- In addition to this, multiple instances of the same Lambda function can be provisioned and run at the same time.
- **You Pay For What You Need:** Another great advantage of using AWS Lambda is that you only pay for what you need. Lambda can accommodate applications with differing loads and charges you for the number of requests your Lambda functions receive and the time it takes to execute those requests per 100ms.

Disadvantages of AWS Lambda:

- **Cold Start:** The serverless architecture executes functions on temporarily created containers. Let's consider a simple function, such as a client registering their details. When the client submits their information to the Lambda function, AWS will create a temporary container, deploy all the dependencies involved, and run the required code. Once the request is completed, the container is destroyed. Now, if there is another request after some time, the same process will be followed. The drawback lies in the time taken by Lambda to create this temporary container. This is usually between 100 milliseconds to 2 minutes and is known as a cold start. However, there is a workaround that we recently implemented in our application. You must ping your Lambda every 5 minutes to ensure the container is not destroyed. This way, only the first request will take some time to process, and all the subsequent ones will be processed much faster without delay.
- **Computational Restrictions:** The functions that require a lot of processing cannot be handled by AWS Lambda. However, there is a workaround here as well. You can create a queuing process to break up your work into a series of smaller functions and execute them simultaneously using AWS Lambda.
- **Vendor control and vendor lock-in:** Another issue with AWS Lambda is that the third-party applications used to get are decided by AWS. This means you would have to give up much control over your application. Another pain point is vendor lock-in. Once the operations are built around a serverless application, it can be costly, time-consuming, and difficult to port your operations elsewhere.
- **Issues with working in a Virtual Private Cloud:** Many clients use a Virtual Private Cloud (VPC) for an extra layer of security. Using Lambda with any such function can entail some additional delay over and above the cold start mentioned above. This, too, can be dealt with using a warm start.

Alternatives:

- Google App Engine.
- Salesforce Heroku.
- Azure App Service.
- Cloud Foundry.
- Salesforce Platform.
- PythonAnywhere.
- Azure Functions.

Steps in the Transformation Layer :

1. Ensure the Kinesis Firehose is properly configured to receive real-time data from the API via Kinesis Data Streams.
2. Create Lambda function using AWS Lambda Service.
 - a. Clearly state the purpose of your Lambda function when naming the lambda function (e.g convert_to_csv, converting data stream logs into CSV)
 - b. Choose the programming language (e.g., Python, Node.js).

- c. Write your Lambda function code in the inline code editor to transform real time data streams

Function name

Enter a name that describes the purpose of your function.

convert_to_csv

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.10

Architecture Info

Choose the instruction set architecture you want for your function code.

☒ x86_64
 ☐ arm64

Permissions Info

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

► Change default execution role

► Advanced settings

Cancel

Create function

into small partitions of csv files. Deploy the code and it will create lambda function.

Code

Test

Monitor

Configuration

Aliases

Versions

Code source Info

File

Edit

Find

View

Go

Tools

Window

Test

Deploy

Changes not deployed

Go to Anything (Ctrl-P)

Environment

convert_to_csv

lambda_function.py

```

1 from __future__ import print_function # Import print_function to ensure compatibility with Python 2 and 3
2
3 import base64 # Import base64 module for encoding and decoding operations
4 import re # Import re module for regular expressions (not used in this example)
5 import json # Import json module for JSON handling
6 print('Loading function') # Print a loading message when the Lambda function starts
7
8 def lambda_handler(event, context):
9     output = [] # Initialize an empty list to store transformed records
10    succeeded_record_cnt = 0 # Initialize a counter for successful record processing
11    failed_record_cnt = 0 # Initialize a counter for failed record processing
12
13    # Iterate through each record in the event['records'] list
14    for record in event['records']:
15        print(record['recordId']) # Print the unique identifier of the current record
16
17        print(record['data']) # Print the base64 encoded data received in the current record
18

```

- d. Go to configurations in your Kinesis Firehose and Link the Lambda function to Firehose to enable data transformation before loading the processed data into the S3 bucket.

Transform source records with AWS Lambda Info

Amazon Data Firehose can invoke an AWS Lambda function to transform, filter, decompress, convert and process your source data records. The specified AWS Lambda function can also be used to provide data partitioning keys for the incoming source data before its delivery to the specified destination.

☒ Turn on data transformation

AWS Lambda function

Version or alias

Choose a Lambda function or enter an ARN

Choose a version...

Browse

Create function

Format: am:aws:lambda:[Region]:[AccountId]:function:[FunctionName]

Buffer size

The AWS Lambda function has a 6 MB invocation payload quota. Your data can expand in size after it's processed by the AWS Lambda function. A smaller buffer size allows for more room should the data expand after processing.

1 MB

Minimum: 0.2 MB, maximum: 3 MB.

Buffer interval

The period of time during which Amazon Data Firehose buffers incoming data before invoking the AWS Lambda function. The AWS Lambda function is invoked once the value of the buffer size or the buffer interval is reached.

60 seconds

Minimum: 0 seconds, maximum: 900 seconds.

Code Flow

```
import base64 # Import base64 module for encoding and decoding operations
import re # Import re module for regular expressions (not used in this example)
import json # Import json module for JSON handling
print('Loading function') # Print a loading message when the Lambda function starts

def lambda_handler(event, context):
    output = [] # Initialize an empty list to store transformed records
    succeeded_record_cnt = 0 # Initialize a counter for successful record processing
    failed_record_cnt = 0 # Initialize a counter for failed record processing

    # Iterate through each record in the event['records'] list
    for record in event['records']:
        print(record['recordId']) # Print the unique identifier of the current record
        print(record['data']) # Print the base64 encoded data received in the current record
```

```
# Decode the base64 encoded data to a readable payload
payload = base64.b64decode(record['data'])

print(payload) # Print the decoded payload (in binary form)

# Lambda function to convert JSON string to CSV format
to_csv = lambda x: ','.join(map(str, json.loads(x).values()))

# Convert the JSON string payload to CSV format using the lambda function
output_payload = to_csv(payload)

# Append a newline character to the CSV data
output_payload = output_payload + "\n"
```

```
# Prepare the output record with the transformed data
output_record = {
    'recordId': record['recordId'], # Assign the recordId of the current record
    'result': 'Ok', # Indicate that the processing result is 'Ok'
    'data': base64.b64encode(output_payload) # Encode the transformed data back to base64
}

print(output_record) # Print the transformed output record

output.append(output_record) # Append the transformed record to the output list

# Print a completion message with the count of successful and failed records
print('Processing completed. Successful records {}, Failed records {}'.format(succeeded_record_cnt, failed_record_cnt))

# Return the transformed records as part of the response
return {'records': output}
```

OUTPUT IN DESTINATION S3 BUCKET

	KDS-S3-jYy6A-1-2024-07-10-08-56-57-a2a20183-ce69-4da6-a799-0e3fb0855a9d	-	July 10, 2024, 14:31:59 (UTC+05:30)	14.2 KB	Standard
---	---	---	---	---------	----------

Overview:

3. Now Extract the Raw Data from the S3 bucket and load that into Snowflake
4. Create SQL Worksheet in Snowflake and configure a database to the sheet.
5. Follow these steps to clean the raw data coming from S3 Bucket.
 - a. Create a table in the database and ingest the 'S3 BUCKET' raw data via Snowpipe to automate the process of data ingestion
 - b. Clean Raw Data and then load it into another table 'CLEAN_TABLE' by Snowpark sessions written in Python script
 - c. Deploy Snowpark sessions into stored procedures
 - d. Create a task to automate the stored procedures and schedule it to trigger daily at a specified interval
 - e. Configure the Snowflake server with Power BI to utilize the cleaned data for reporting and analysis
 - f. Develop Power BI dashboards and reports utilizing cleaned data from the Snowflake database to derive meaningful insights

Code Flow

/* 1. Create or replace a stage (s3_stage) pointing to an S3 bucket, you typically specify the URL of the bucket and credentials for accessing it.

```
CREATE OR REPLACE STAGE s3_stage
URL = 's3://destination-
bucket' CREDENTIALS = ''
AWS_KEY_ID=' '
AWS_SECRET_KEY
);
```

/* 2. Create a table (for raw data) to store data from the 'S3 bucket' data into the Snowflake database.*/

```
DROP TABLE IF EXISTS RAW_DATA;
CREATE TABLE RAW_DATA (
`Country` VARCHAR(50),
`MktName` VARCHAR(50),
`ITEM` VARCHAR(50),
`PRICE` NUMBER(38,0)
);
```

/* 3. Creating or replacing a file format such as csv_format for CSV

a. Create File format to make Snowflake know the format of data file

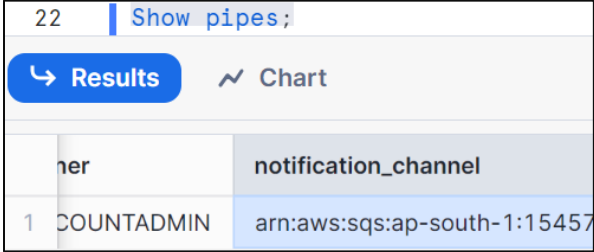
```
CREATE OR REPLACE FILE FORMAT
csv_format TYPE = 'CSV'
FIELD_OPTIONALLY_ENCLOSED_BY = ''
```

/* 4. To automate the ingestion of data into Snowflake from an external stage, you can create a Snowpipe.

```
CREATE OR REPLACE PIPE my_snowpipe
AUTO_INGEST = TRUE
AS
COPY INTO RAW_FOOD_DATA
FROM @s3_stage
FILE_FORMAT = (FORMAT_NAME = csv_format);
Show pipes;
```

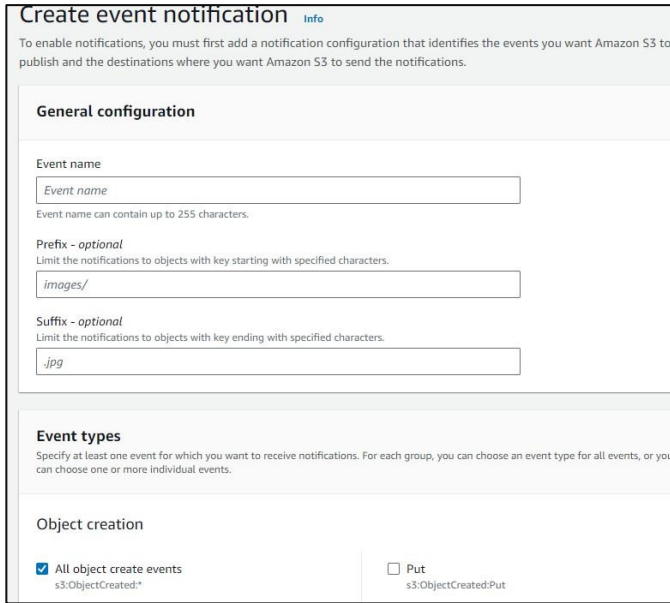
/* 5. Automate Data Loading from S3 to Snowflake Using Snowpipe with SQS

- Execute the SHOW PIPES command in Snowflake to retrieve the Snowpipe details, including the notification channel ARN.



	name	notification_channel
1	COUNTADMIN	arn:aws:sqs:ap-south-1:15457

- Copy the ARN of the notification channel for use in S3 event notifications.
- In the properties section of your destination S3 Bucket, configure event notifications to send messages to the SQS queue when new objects/ data is created in S3 Bucket(e.g., s3:ObjectCreated:*).



Create event notification [Info](#)

To enable notifications, you must first add a notification configuration that identifies the events you want Amazon S3 to publish and the destinations where you want Amazon S3 to send the notifications.

General configuration

Event name

 Event name can contain up to 255 characters.

Prefix - optional
 Limit the notifications to objects with key starting with specified characters.

Suffix - optional
 Limit the notifications to objects with key ending with specified characters.

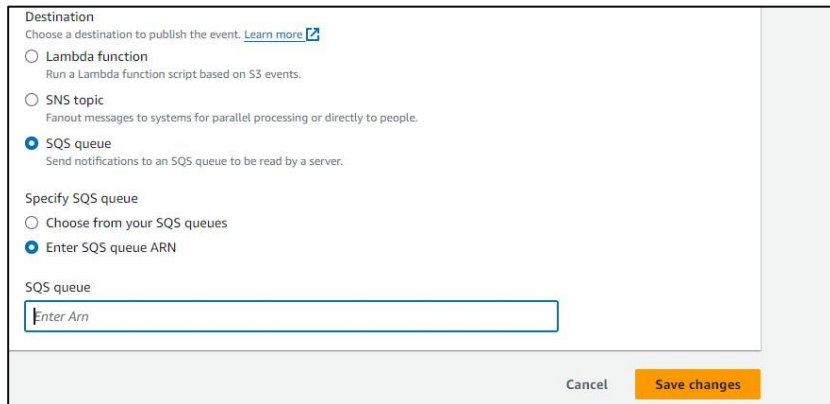
Event types
 Specify at least one event for which you want to receive notifications. For each group, you can choose an event type for all events, or you can choose one or more individual events.

Object creation

☒ All object create events
 s3:ObjectCreated:*

☐ Put
 s3:ObjectCreated:Put

- Use the copied ARN to configure the Snowpipe to listen for events from the SQS queue.



Destination
 Choose a destination to publish the event. [Learn more](#)

☐ Lambda function
 Run a Lambda function script based on S3 events.

☐ SNS topic
 Fanout messages to systems for parallel processing or directly to people.

☒ SQS queue
 Send notifications to an SQS queue to be read by a server.

Specify SQS queue

☐ Choose from your SQS queues

☒ Enter SQS queue ARN

SQS queue

Cancel **Save changes**

/*6. Row_count table tracks incremental data additions for monitoring and managing data transformations in Snowflake.

```
create or replace table row_count (  
  
    // AUTOINCREMENT, starting at 1 and incrementing by 1 for each new row. This serves  
    as a unique identifier for each row in the table.  
  
    index_no number autoincrement start 1 increment 1,  
    no_of_rows bigint,  
    date varchar(100),  
    no_of_rows_added bigint,  
    no_of_rows_in_transformed_table bigint  
);
```

/* 7. Creating Snowpark Sessions in separate worksheet for Data Cleaning in Python

```
// The Snowpark package is required for Python Worksheets.  
// You can add more packages by selecting them using the Packages control and  
then importing them.  
  
// Import Snowpark for Snowflake operations  
import snowflake.snowpark as snowpark  
  
// Import specific functions from Snowpark  
from snowflake.snowpark.functions import col  
  
// Import pandas for DataFrame manipulation  
import pandas as pd  
  
# Main function to handle data processing using the provided Snowpark session.  
def main(session: snowpark.Session):  
    tableName = 'raw_food_data' # Name of the input table containing raw data  
  
    # Load the raw data table into a Snowpark DataFrame  
    snow_table = session.table(tableName)
```

```
# Convert the Snowpark DataFrame to a pandas DataFrame for easier manipulation
df = snow_table.to_pandas()

# Print the shape of the DataFrame to understand its dimensions
print(df.shape)

# Name of the table that tracks row counts
tableName = 'row_count'

# Load the row count table into a Snowpark DataFrame
row_count_table = session.table(tableName)

# Convert the row count table to a pandas DataFrame
row_count_df = row_count_table.to_pandas()

if row_count_df.shape[0] != 0:

    # If there are existing rows in the row count table, get the latest count
    row_count_df = row_count_df[row_count_df['INDEX_NO'] ==
max(row_count_df['INDEX_NO'])].reset_index(drop=True)
    old_rows = row_count_df['NO_OF_ROWS'][0] # Get the number of rows
from the latest record
    df = df.sort_values(by='YEAR').reset_index(drop=True) # Sort the raw data
by year
    df1 = df.tail(df.shape[0] - old_rows) # Get only the new rows added since
the last count
else:
    df1 = df # If no prior data, use the entire DataFrame as new data

print('Shape 1:', df1.shape) # Print the shape of the new DataFrame
print('Shape 1:', df1['DATES'].unique()) # Print unique dates in the new
DataFrame

# Select all columns that have at least one non-null value
df_notnull = df1.loc[:, df1.notnull().any()]

# Define columns to keep for further processing
keep_cols = ['COUNTRY', 'MKT_NAME', 'DATES']

# Filter out columns not in the keep_cols list while keeping relevant
columns
df_1 = df_notnull[keep_cols + [col for col in df_notnull.columns if
col.startswith(('O_', 'H_', 'L_', 'C_', 'INFLATION_', 'TRUST_'))]]

# Convert the DataFrame to a desired long format
final_component_df = df_1.melt(id_vars=keep_cols, var_name='ITEM',
value_name='VALUE')
```

```
# Split the variable names to get the type of data (open, high, low, close,
inflation, trust)
final_component_df.insert(loc=4, column='TYPE',
value=final_component_df['ITEM'].str.split('_').str[0])

final_component_df['TYPE'] = final_component_df['TYPE'].replace({'O':
'OPEN', 'H': 'HIGH', 'L': 'LOW', 'C': 'CLOSE'})

# Extract the item names from ITEM column
final_component_df['ITEM'] = final_component_df['ITEM'].str.split('_').str[1]

# Pivot the DataFrame to have a structured format with different types as
columns
pivot_type_df = final_component_df.pivot_table(index=['COUNTRY',
'MKT_NAME', 'DATES', 'ITEM'], values='VALUE', columns='TYPE').reset_index()

# Define expected columns
present_cols = pivot_type_df.columns.tolist() # Get the list of current
columns
expected_cols = ['COUNTRY', 'MKT_NAME', 'DATES', 'ITEM', 'CLOSE', 'HIGH',
'INFLATION', 'LOW', 'OPEN', 'TRUST']

# Identify any missing columns and add them with default value 0
add_cols = list(set(expected_cols) - set(present_cols))
if add_cols:
    for col in add_cols:
        pivot_type_df[col] = 0 # Add missing columns initialized to 0

# Select only the expected columns for final DataFrame
final_df = pivot_type_df[expected_cols]

# Print shape of the cleaned DataFrame
print('Shape CLEANSED_FOOD_DATA_NEW:', pivot_type_df.shape)
print(pivot_type_df['DATES'].unique()) # Print unique dates in the cleaned
DataFrame

# If the row count table is empty, create a new entry
if row_count_df.shape[0] == 0:
    d = {
        'INDEX_NO': 1,
        'NO_OF_ROWS': df.shape[0],
        'DATE': str(pd.Timestamp.now()),
        'NO_OF_ROWS_ADDED': df.shape[0],
        'NO_OF_ROWS_IN_TRANSFORMED_TABLE': pivot_type_df.shape[0]
    }
    session.create_dataframe(pd.DataFrame([d])).write.save_as_table('row_count',
mode="append", table_type="")
else:
```

```
# If rows exist, increment the index number and update counts
d = {
    'INDEX_NO': row_count_df['INDEX_NO'][0] + 1,
    'NO_OF_ROWS': df.shape[0],
    'DATE': str(pd.Timestamp.now()),
    'NO_OF_ROWS_ADDED': int(df.shape[0] - old_rows),
    'NO_OF_ROWS_IN_TRANSFORMED_TABLE': pivot_type_df.shape[0]
}
session.create_dataframe(pd.DataFrame([d])).write.save_as_table('row_count',
mode="append", table_type="")

# Create a Snowpark DataFrame for the final cleaned data
snow_component = session.create_dataframe(final_df)

# Save the cleaned data into a Snowflake table
snow_component.write.save_as_table("CLEANSED_FOOD_DATA_NEW",
mode="append")

# Return the cleaned DataFrame to appear in the Results tab
return snow_component

# Connection parameters for Snowflake
connection_parameters = {
    "account": "<your_account>", # Your Snowflake account identifier
    "user": "<user_name>", # Your Snowflake username
    "password": "<your_password>", # Your Snowflake password
    "role": "ACCOUNTADMIN", # Optional role for the session
    "warehouse": "COMPUTE_WH", # warehouse for the session
    "database": "<your_database>", # database to connect to
    "schema": "PUBLIC", # schema to use
}
```

/* 7. Run Snowpark Session script to ensure that the cleaned data is produced with only the necessary columns, confirming the integrity and relevance of the data.

	COUNTRY	MKT_NAME	DATES	ITEM	CLOSE	HIGH
1	Armenia	Armavir	2009-01-01	APPLES	1159.05	1173.84
2	Armenia	Armavir	2009-01-01	BREAD	386.94	389.87
3	Armenia	Armavir	2009-01-01	CABBAGE	142.68	146.95
4	Armenia	Armavir	2009-01-01	CARROTS	237.87	240.95
5	Armenia	Armavir	2009-01-01	CHEESE	1599.85	1612.63
6	Armenia	Armavir	2009-01-01	CUCUMBERS	906.46	957.89

/* 8. Deploy the Snowpark sessions into stored procedures, encapsulating the data cleaning logic within a structure.



```

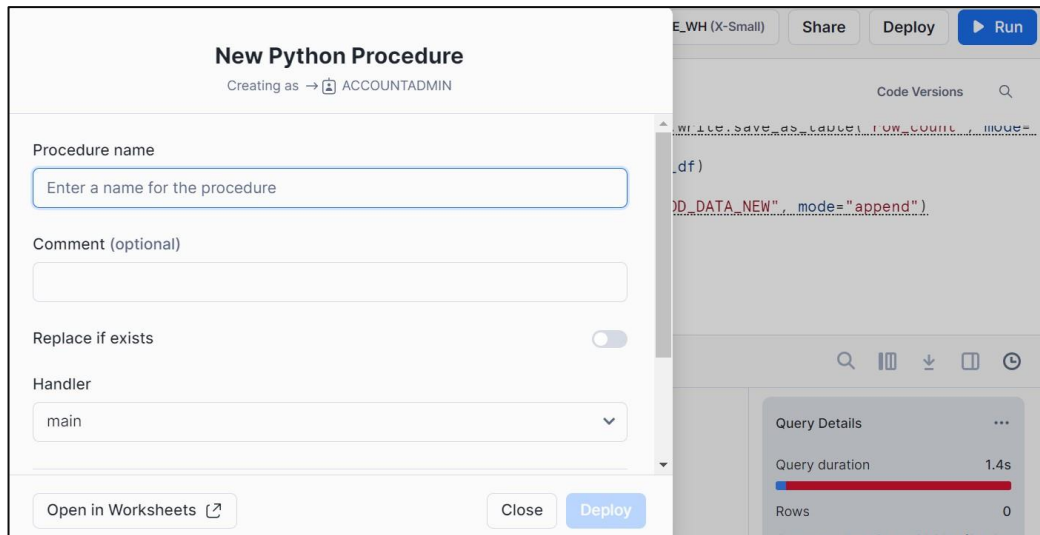
ACCOUNTADMIN • COMPUTE_WH (X-Small) Share Deploy Run

Settings Packages Code Versions

// Create dataframe(pu.DataFrame(10)).write.save_as_table('ROW_COUNT', mode=
ment = session.create_dataframe(final_df)
ment.write.save_as_table("CLEANSED_FOOD_DATA_NEW", mode="append")

alue will appear in the Results tab.
w_component
  
```

- a) Select the “Handle” as main , it acts as the primary entry point for executing the stored procedure, making it clear where the execution starts.



New Python Procedure
Creating as → ACCOUNTADMIN

Procedure name
Enter a name for the procedure

Comment (optional)

Replace if exists ☐

Handler
main

Open in Worksheets [↗](#) Close Deploy

Query Details
Query duration 1.4s
Rows 0

/* 9. Create tasks to schedule the stored procedures to automate the execution at specified intervals, ensuring that data cleaning occurs regularly without manual intervention.

```
48      --- Create the Task which calls the procedure
49
50      create or replace task transform_schema
51          warehouse = COMPUTE_WH
52          schedule = 'USING CRON 0 12 * * * UTC' --schedule daily at 12 PM UTC
53      as
54          call stored_procedures()
55
56      --start the task
57      alter task transform_schema resume;
```

This code snippet does the following:

- a. **Create a Task:** It defines a new task named `transform_schema` in Snowflake.
- b. **Specify Warehouse:** The task will use the `COMPUTE_WH` warehouse for execution, which provides the compute resources needed to run the task.
- c. **Set Schedule:** The task is scheduled to run daily at 12 PM UTC using a CRON expression.
- d. **Call Stored Procedure:** When the task executes, it will call the stored procedure `stored_procedures()`.
- e. **Start the Task:** The command `alter task transform_schema resume;` activates the task, allowing it to begin executing according to the defined schedule.

OUTPUT :

Results							Chart		Query Details	
	COUNTRY	MKT_NAME	DATES	ITEM	CLOSE	HIGH	Query duration			561ms
1	Armenia	Armavir	2009-01-01	APPLES	1159.05	1173.84	Rows			1.4K
2	Armenia	Armavir	2009-01-01	BREAD	386.94	389.87				
3	Armenia	Armavir	2009-01-01	CABBAGE	142.68	146.95				

“

Did you Know?

Enhanced fan-out is an Amazon Kinesis Data Streams feature that enables consumers to receive records from a data stream with a dedicated throughput of up to 2 MB of data per second per shard. A consumer using enhanced fan-out doesn't have to contend with other consumers receiving data from the stream.

For a streaming application of unbounded data sets, the completeness of all incoming data is crucial. To guarantee that every data is processed, you can use watermarks in Flink applications to track the events within a given timeframe(window).

Did you Know?

When a windowed query processes each window in a non-overlapping manner.

Real-Time layer

Unique Services used in the Real-Time Layer

AWS Lambda:

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the computing time you consume—there is no charge when your code is not running. With Lambda, you can run code for virtually any application or backend service—all with zero administration. Just upload your code, and Lambda takes care of everything required to run and scale your code with high availability. You can set up your code to trigger automatically from other AWS services or call it directly from any web or mobile app.

Advantages of AWS Lambda:

Many of the advantages of AWS Lambda relate to the advantages of adopting a serverless approach.

- **Saves Time and Effort:** One of the significant benefits of going serverless using AWS Lambda is the time and effort saved from creating and maintaining your application's infrastructure.
- AWS saves a lot of effort by provisioning and managing the infrastructure your Lambda functions run on.
- It automatically scales the instances to handle excessive load times and implements a proper logging and error-handling system. Those involved in constructing or maintaining infrastructure will reasonably understand the gravity of this advantage.
- Not only does it save you time while building the application, but it will also save you ample time required to maintain an already established system as your application evolves and scales up.
- Lambda ensures quicker production and greater agility in launching your product in the market, thus giving you an edge over your competition.
- **Helps in building a scalable solution:** AWS Lambda has become one of the most popular solutions in such a short span of time because of its ability to accommodate applications at scale and early stages.
- For large loads of applications, AWS runs your Lambda function simultaneously with other Lambda functions. This means that you do not have to worry about clogged-up queues.
- In addition to this, multiple instances of the same Lambda function can be provisioned and run at the same time.
- **You Pay For What You Need:** Another great advantage of using AWS Lambda is that you only pay for what you need. Lambda can accommodate applications with differing loads and charges you for the number of requests your Lambda functions receive and the time it takes to execute those requests per 100ms.

Disadvantages of AWS Lambda:

- **Cold Start:** The serverless architecture executes functions on temporarily created containers. Let's consider a simple function, such as a client registering their details. When the client submits their information to the Lambda function, AWS will create a temporary container, deploy all the dependencies involved, and run the required code. Once the request is completed, the container is destroyed. Now, if there is another request after some time, the same process will be followed. The drawback lies in the time taken by Lambda to create this temporary container. This is usually between 100 milliseconds to 2 minutes and is known as a cold start. However, there is a workaround that we recently implemented in our application. You must ping your Lambda every 5 minutes to ensure the container is not destroyed. This way, only the first request will take some time to process, and all the subsequent ones will be processed much faster without delay.
- **Computational Restrictions:** The functions that require a lot of processing cannot be handled by AWS Lambda. However, there is a workaround here as well. You can create a queuing process to break up your work into a series of smaller functions and execute them simultaneously using AWS Lambda.
- **Vendor control and vendor lock-in:** Another issue with AWS Lambda is that the third-party applications used to get are decided by AWS. This means you would have to give up much control over your application. Another pain point is vendor lock-in. Once the operations are built around a serverless application, it can be costly, time-consuming, and difficult to port your operations elsewhere.
- **Issues with working in a Virtual Private Cloud:** Many clients use a Virtual Private Cloud (VPC) for an extra layer of security. Using Lambda with any such function can entail some additional delay over and above the cold start mentioned above. This, too, can be dealt with using a warm start.

Alternatives:

- Google App Engine.
- Salesforce Heroku.
- Azure App Service.
- Cloud Foundry.
- Salesforce Platform.
- PythonAnywhere.
- Azure Functions.

Definition: Amazon Kinesis Data Firehose is a fully managed service provided by AWS that facilitates the delivery of streaming data to various AWS data stores and analytics services in near real-time, without the need for custom coding.

Detailed Explanation of Amazon Kinesis Data Firehose

Data Delivery: Kinesis Data Firehose simplifies the process of ingesting and loading streaming data into destinations such as Amazon S3, Amazon Redshift, Amazon Elasticsearch Service, and Splunk. It automatically scales to match the throughput of incoming data and ensures reliable delivery to these destinations.

Integration with AWS Services: It integrates seamlessly with other AWS services, allowing you to leverage the capabilities of services like S3 for durable storage, Redshift for data warehousing, Elasticsearch for real-time search and analytics, and Splunk for log analysis.

Transformation and Compression: Kinesis Data Firehose can transform data before delivering it to the destination. It supports data transformation using AWS Lambda functions, which allows you to modify the data format, filter records, or mask sensitive information before storing or analyzing it.

Monitoring and Management: The service provides built-in monitoring through Amazon CloudWatch, which helps track the volume of incoming data, delivery success rates, and potential errors. It also supports Amazon CloudTrail for logging API calls made to the service.

Security and Compliance: Kinesis Data Firehose ensures data security by encrypting data in transit using HTTPS and allowing you to encrypt data at rest using AWS KMS. It supports compliance with various regulations and standards, including HIPAA, GDPR, and PCI DSS.

Advantages of Amazon Kinesis Data Firehose

- **Simplicity:** Eliminates the need for managing infrastructure and writing custom code, making it easy to set up and use for streaming data ingestion.
- **Scalability:** Automatically scales to handle varying data volumes, ensuring reliable and efficient delivery to destinations without manual intervention.
- **Integration:** Seamless integration with AWS data stores and analytics services provides flexibility in choosing where to store and analyze data.
- **Real-Time Processing:** Facilitates near real-time data processing and analytics, enabling timely insights and decision-making.

Disadvantages of Amazon Kinesis Data Firehose

- **Limited Transformation Options:** While it supports basic data transformations through AWS Lambda, more complex data processing tasks may require additional services or custom implementations.
- **Cost Considerations:** Costs can accumulate based on data volume and the chosen destination service, necessitating careful monitoring and cost management.

""Did you know

Amazon Kinesis Data Firehose is touted as a "no-code" solution primarily because it abstracts away the complexities of managing and writing custom code for streaming data ingestion and delivery. Here's why it is considered a no-code solution:

1. **Managed Service:** *Kinesis Data Firehose is fully managed by AWS, which means AWS takes care of infrastructure provisioning, scaling, and maintenance. Users do not need to set up or manage servers, databases, or other infrastructure components.*
2. **Configuration-Based Setup:** *Setting up Kinesis Data Firehose involves configuring data sources (like Kinesis Data Streams), specifying the destination (e.g., S3, Redshift), and optionally applying transformations using AWS Lambda functions. These configurations are done through the AWS Management Console, CLI, or SDKs without writing code.*
3. **Automated Scalability:** *The service automatically scales to handle varying data volumes and delivery requirements. Users do not need to manually adjust resources or write code to manage scalability.*

Amazon S3 Bucket

Amazon S3 (Simple Storage Service) is a cloud-based object storage service provided by AWS. It allows users to store and retrieve data from anywhere on the web, offering scalability, high durability (99.999999999%), and availability. S3 stores data as objects within buckets, accessible via unique URLs. It supports various storage classes for different access and cost requirements, including Standard, Intelligent-Tiering, Glacier for archival, and others. Security features include encryption options for data at rest and in transit, access control through IAM policies, and auditing with AWS CloudTrail. S3 integrates with AWS services and third-party applications, supporting diverse use cases like data backup, analytics, and content distribution. Its pay-as-you-go pricing model ensures cost efficiency, billing based on storage usage, data transfers, and requests.

Key Features of Amazon S3:

1. **Scalability:** S3 scales automatically to accommodate growing data volumes, making it suitable for applications with unpredictable storage requirements.
2. **Durability and Availability:** It offers 99.999999999% (11 nines) durability by redundantly storing data across multiple facilities and availability zones (AZs), ensuring high availability.
3. **Security:** S3 provides robust security features such as encryption (both at rest and in transit), access controls using AWS Identity and Access Management (IAM), and integration with AWS CloudTrail for logging API calls.
4. **Lifecycle Policies:** Automates the transition of objects between different storage classes (e.g., Standard, Glacier) based on predefined rules, optimizing cost and performance.
5. **Versioning:** Supports versioning of objects, allowing retrieval of previous versions and preventing accidental deletion or overwriting of data.

Advantages of Amazon S3:

- **Versatility:** Suitable for a wide range of use cases including backup and restore, big data analytics, content distribution, and archival.
- **Economical:** Pay-as-you-go pricing model with no upfront costs, allowing organizations to scale storage costs with business needs.

Disadvantages of Amazon S3:

- **Costs:** Data transfer and storage costs can accumulate, especially with frequent access or large volumes of data.
- **Complexity:** Managing permissions, versioning, and lifecycle policies can be complex, requiring careful configuration and monitoring.

Alternatives to Amazon S3:

- **Google Cloud Storage (GCS):** Similar object storage service provided by Google Cloud Platform.
- **Microsoft Azure Blob Storage:** Object storage solution offered by Microsoft Azure with similar features to Amazon S3.

Amazon Kinesis Data Streams

It is a fully managed service provided by AWS that enables real-time processing of streaming data at scale. It allows you to ingest, process, and analyze large volumes of streaming data in real time.

How Kinesis Data Streams Works

Handling Latency and Throughput: Kinesis Data Streams handles high throughput and low-latency streaming data through the following mechanisms:

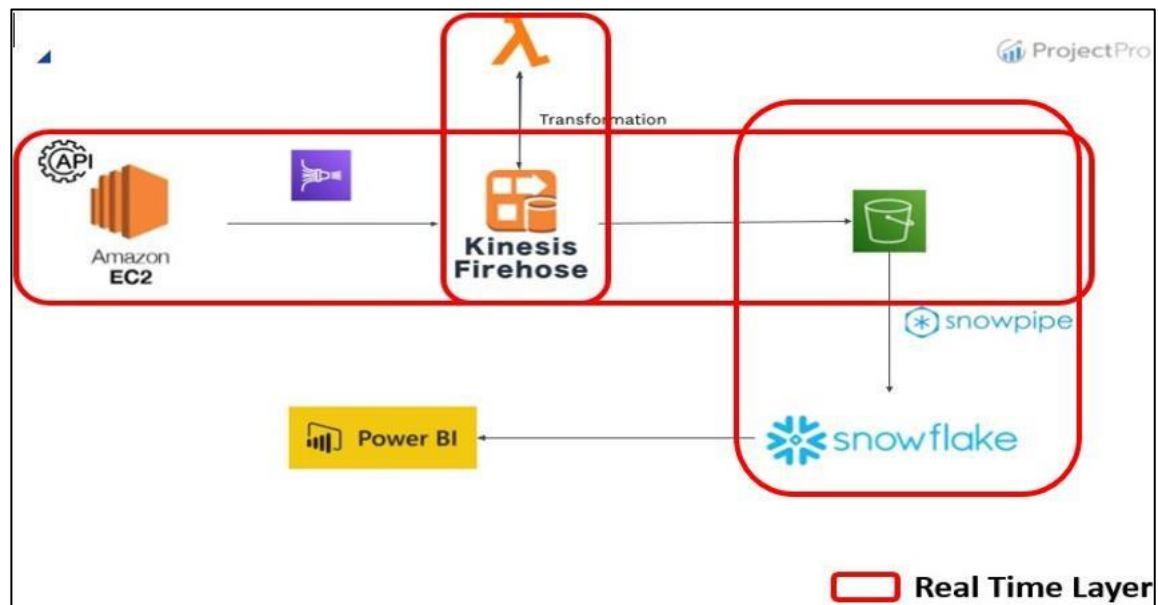
- **Sharding:** Data is partitioned into shards, which are the basic units of throughput in Kinesis. Each shard can ingest up to 1,000 records per second or 1 MB per second, allowing for horizontal scaling as data volume increases.
- **Data Retention:** Streams store data durably for a specified retention period (default 24 hours, extendable up to 7 days) across multiple Availability Zones (AZs). This ensures fault tolerance and data resilience.
- **Scalability:** You can dynamically adjust the number of shards in a stream to handle changes in data ingestion rates. This elasticity allows Kinesis to adapt to varying workloads and bursty data spikes.

Key Features of Kinesis Data Streams

- **Real-time Processing:** Facilitates real-time analytics and decision-making by processing data streams with minimal latency.
- **Integration:** Seamlessly integrates with other AWS services such as Lambda, Kinesis Data Analytics, and Kinesis Data Firehose, as well as third-party applications, enabling flexible and extensible data processing pipelines.
- **Monitoring and Metrics:** Provides built-in monitoring tools and metrics to monitor the health, performance, and consumption of data streams, helping you optimize and troubleshoot your streaming applications.
- **Security:** Implements encryption of data in transit and at rest, access control through AWS Identity and Access Management (IAM), and compliance with industry standards (e.g., GDPR, HIPAA) to ensure secure handling of sensitive data.
- **Use Cases:** Commonly used for real-time analytics, IoT device telemetry, log and event processing, clickstream analysis, and more, offering a robust solution for streaming data management in cloud environments.

Amazon Kinesis Data Streams empowers organizations to build scalable and resilient streaming data architectures, supporting a wide range of real-time use cases with high reliability and performance.

Real-Time Layer:

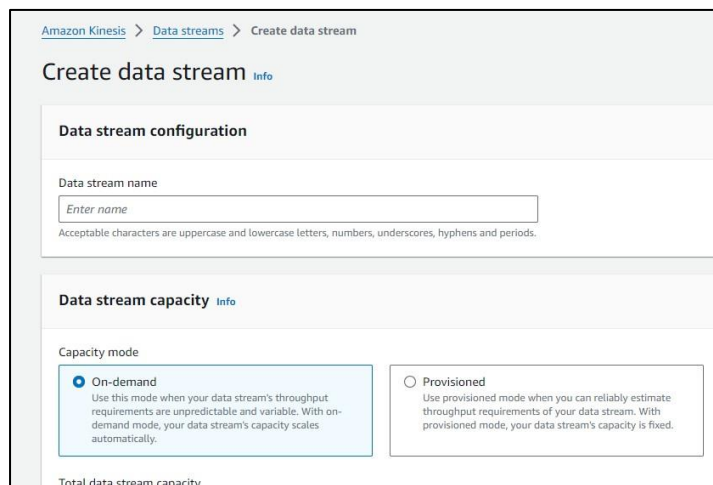


Steps in the Real-Time Layer:

1. Create a Kinesis Client

```
#Initialize Kinesis Client
import boto3
kinesis_client = boto3.client('kinesis', region_name='ap-south-1', aws_access_key_id='',
                               aws_secret_access_key='')
```

2. Create Kinesis Data Streams using AWS Kinesis Service



The screenshot shows the AWS Management Console 'Create data stream' page. The page has a breadcrumb trail: Amazon Kinesis > Data streams > Create data stream. The main heading is 'Create data stream' with an 'Info' link. Below this is the 'Data stream configuration' section. It contains a 'Data stream name' field with a placeholder 'Enter name' and a note: 'Acceptable characters are uppercase and lowercase letters, numbers, underscores, hyphens and periods.' Below this is the 'Data stream capacity' section with an 'Info' link. It contains a 'Capacity mode' section with two radio buttons: 'On-demand' (selected) and 'Provisioned'. The 'On-demand' option has a description: 'Use this mode when your data stream's throughput requirements are unpredictable and variable. With on-demand mode, your data stream's capacity scales automatically.' The 'Provisioned' option has a description: 'Use provisioned mode when you can reliably estimate throughput requirements of your data stream. With provisioned mode, your data stream's capacity is fixed.' At the bottom, there is a 'Total data stream capacity' label.

3. Make an HTTP request to pull data from API endpoints

Data Flow: Data is streamed from API endpoints to Kinesis Data Streams via the Kinesis Client.

```
def send_to_kinesis(data):
    # Send data to Kinesis stream
    data_record = json.dumps(data)
    kinesis_client.put_record(StreamName='my-kinesis-stream', Data=data_record, PartitionKey='1')

def invoke_api():
    # API Gateway endpoint URL

    api_endpoint = 'https://vf23h36ius.us-east-2.amazonaws.com/fetch_data'
    # Define the parameters
    params = {
        'year': '2007',
    }
    # Make a GET request to the API
    response = requests.get(api_endpoint, params=params)

    json_data = json.loads(response.text)
```

4. Set up Kinesis Firehose to ingest data from Kinesis Data Streams

Data Flow: Kinesis Data Streams send real-time data to Kinesis Firehose.

Choose source and destination

Specify the source and the destination for your Firehose stream. You cannot change the source and destination of your Firehose stream once it has been created.

Source

Info

Choose a source

Destination

Info

Choose a destination

Cancel

Create Firehose stream

5. Create a Lambda function and attach it to Kinesis Firehose

Data Flow: Kinesis Firehose invokes the Lambda function to transform the data.

Lambda Function

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Info

File

Edit

Find

View

Go

Tools

Window

Test

Deploy

Changes not deployed

Go to Anything (Ctrl-P)

lambda_function

Environment Var

Environment

convert_to_csv

lambda_function.py

```
1 from __future__ import print_function # Import print_function to ensure compatibility with Python 2 and 3
2 import base64 # Import base64 module for encoding and decoding operations
3 import re # Import re module for regular expressions (not used in this example)
4 import json # Import json module for JSON handling
5 print('Loading function') # Print a loading message when the Lambda function starts
6
7
8 def lambda_handler(event, context):
9     output = [] # Initialize an empty list to store transformed records
10    succeeded_record_cnt = 0 # Initialize a counter for successful record processing
11    failed_record_cnt = 0 # Initialize a counter for failed record processing
12
13    # Iterate through each record in the event['records'] list
14    for record in event['records']:
15        print(record['recordId']) # Print the unique identifier of the current record
16
17        print(record['data']) # Print the base64 encoded data received in the current record
18
```

Configure Lambda Function with Kinesis Data Firehose

Transform source records with AWS Lambda [Info](#)

Amazon Data Firehose can invoke an AWS Lambda function to transform, filter, decompress, convert and process your source data records. The specified AWS Lambda function can also be used to provide data partitioning keys for the incoming source data before its delivery to the specified destination.

☒ Turn on data transformation

AWS Lambda function Version or alias

Format: arn:aws:lambda:[Region]:[AccountId]:function:[FunctionName]

Buffer size
The AWS Lambda function has a 6 MB invocation payload quota. Your data can expand in size after it's processed by the AWS Lambda function. A smaller buffer size allows for more room should the data expand after processing.

MB
Minimum: 0.2 MB, maximum: 3 MB.

Buffer interval
The period of time during which Amazon Data Firehose buffers incoming data before invoking the AWS Lambda function. The AWS Lambda function is invoked once the value of the buffer size or the buffer interval is reached.

seconds
Minimum: 0 seconds, maximum: 900 seconds.

- Create another S3 bucket as the destination for Kinesis Firehose

Data Flow: Transformed data from Kinesis Firehose is stored in the S3 bucket.

Objects (1) [Info](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	10/	Folder	-	-	-

- Create External Stage in Snowflake to Extract Raw Data from S3 Bucket

Data Flow: Snowflake accesses the raw data files stored in the S3 bucket via external stage.

```

1 CREATE OR REPLACE STAGE s3_stage
2     URL = 's3://destination-bucket/data'
3     CREDENTIALS = (
4         AWS_KEY_ID=' '
5         AWS_SECRET_KEY=' '
6     );
7

```

- Create Snowflake Table (staging table) to ingest Raw Data from External Stage

Table Details Columns Data Preview Co

Table definition

```

1 create or replace TABLE
2 (
3     COUNTRY VARCHAR(167
4     MKT_NAME VARCHAR(16
5     DATES DATE,
6     YEAR NUMBER(38,0),
7     MONTH NUMBER(38,0),
8     O_APPLES FLOAT,
9     H_APPLES FLOAT,
10    L_APPLES FLOAT,
11    C_APPLES FLOAT,

```

9. Use the 'COPY INTO' command to ingest data from the external stage into the Snowflake staging table.

Data Flow: Data is ingested from the S3 bucket through the external stage into the Snowflake table.

```
17 COPY INTO RAW_FOOD_DATA
18 FROM @s3_stage
19 FILE_FORMAT = (FORMAT_NAME = csv_format);
20
21
```

10. Create snowpipe to auto ingest the data from s3 bucket into snowflake table

```
13
14 CREATE OR REPLACE PIPE my_snowpipe
15 AUTO_INGEST = TRUE
16 AS
17 COPY INTO RAW_FOOD_DATA
18 FROM @s3_stage
19 FILE_FORMAT = (FORMAT_NAME = csv_format);
20
21
```

- a) Execute show pipe



- b) Copy the Notification channel credential of Snowpipe i.e ARN
- c) Configure your S3 bucket to send notifications to an SQS queue whenever new data files are added.
- d) Create an event notification in the S3 bucket properties and specify the SQS queue ARN as the destination

Create event notification
Info

To enable notifications, you must first add a notification configuration that identifies the events you want Amazon S3 to publish and the destinations where you want Amazon S3 to send the notifications.

General configuration

Event name

Event name can contain up to 255 characters.

Prefix - optional
Limit the notifications to objects with key starting with specified characters.

Suffix - optional
Limit the notifications to objects with key ending with specified characters.

Event types
Specify at least one event for which you want to receive notifications. For each group, you can choose an event type for all events, or you can choose one or more individual events.

Object creation

☒ All object create events
s3:ObjectCreated:*

☐ Put
s3:ObjectCreated:Put

e) Insert Snowpipe Notification Channel ARN into S3 Bucket Configuration.

- Data Flow:** 1. When a new data file is uploaded to the S3 bucket, an event notification is triggered and sent to the specified SQS queue.
2. The data ingestion process begins as soon as new files are uploaded to S3 bucket. S3 sends an event notification to SQS, which Snowflake listens to and processes to ingest the data into Snowflake.

Specify SQS queue
☐ Choose from your SQS queues
☒ Enter SQS queue ARN

SQS queue

Cancel
Save changes

11. Create Snowpark session in python worksheet for data cleaning.

Data Flow: Raw data from the staging table in Snowflake is processed and cleaned using Snowpark sessions. The cleaned data is then stored in a new Snowflake table.

```

1  # The Snowpark package is required for Python Worksheets.
2  # You can add more packages by selecting them using the Packages control and then importing
3
4  import snowflake.snowpark as snowpark
5  from snowflake.snowpark.functions import col
6  import pandas as pd
7
8  def main(session: snowpark.Session):
9      # Your code goes here, inside the "main" handler.
10     tableName = 'raw_food_data'
11
12     snow_table = session.table(tableName)
13
14     # convert table to pandas dataframe
15     df=snow_table.to_pandas()

```

12. Save the cleaned data to a new table in Snowflake, ensuring it is ready for further processing or analysis.

```

81
82     snow_component.write.save_as_table("CLEANSED_FOOD_DATA_NEW", mode="append")
83
84     # Return value will appear in the Results tab.
85     return snow_component
86

```

Output: Cleaned Data

35

select * from CLEANSED_FOOD_DATA_NEW;

36

Results

Chart

🔍

📄

⬇️

📋

🕒

	COUNTRY	MKT_NAME	DATES	ITEM
1	Armenia	Armavir	2009-01-01	APPLES
2	Armenia	Armavir	2009-01-01	BREAD
3	Armenia	Armavir	2009-01-01	CABBAGE
4	Armenia	Armavir	2009-01-01	CARROTS
5	Armenia	Armavir	2009-01-01	CHEESE

Query Details

...

Query duration

523ms

Rows

1.4K

Query ID

01b5a8ef-0000-af24-0...

Show more

▼

13. Deploy Snowpark Sessions into Stored Procedures that encapsulate the Snowpark session code

Data Flow: The Snowpark session script is automatically executed whenever new data is loaded into the staging table, ensuring that data cleaning is performed continuously and without manual intervention.

ACCOUNTADMIN • COMPUTE_WH (X-Small) Share Deploy Run

Settings Packages Code Versions

```

final_df = session.create_dataframe(pd.DataFrame(LUJ)).write.save_as_table('LOW_COUNT', mode='append')

final_df = session.create_dataframe(final_df)

final_df.write.save_as_table("CLEANSED_FOOD_DATA_NEW", mode="append")

The value will appear in the Results tab.
w_component

```

14. Create tasks to automate stored procedures run at specific intervals.

Data Flow:

1. Snowflake tasks are scheduled using a cron expression or a predefined schedule. A task is scheduled to run every hour to process new data.
2. When the task triggers, it executes the associated stored procedure. The stored procedure performs necessary data transformations, such as cleaning, filtering, and aggregating data.

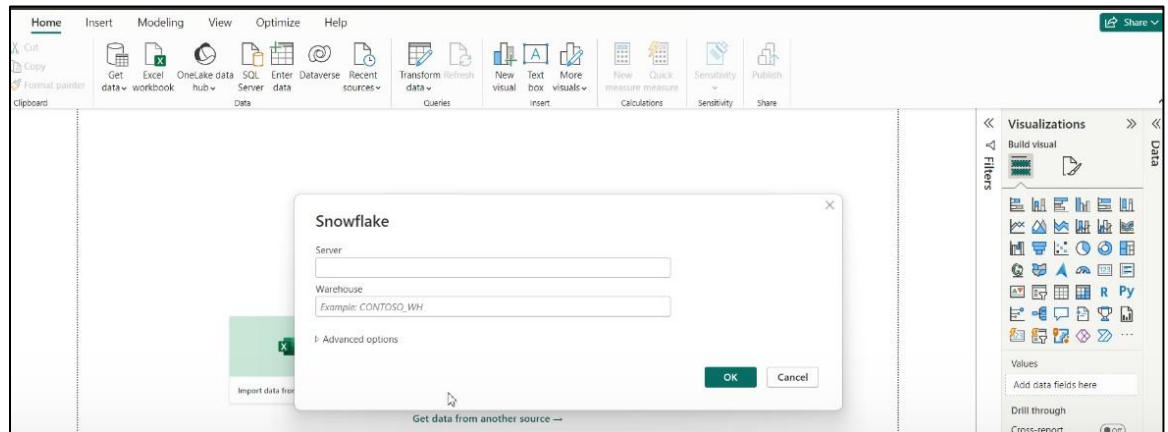
```

48 --- Create the Task which calls the procedure
49
50 create or replace task transform_schema
51   warehouse = COMPUTE_WH
52   schedule = 'USING CRON 0 12 * * * UTC' --schedule daily at 12 PM UTC
53 as
54   call |stored_procedures()
55

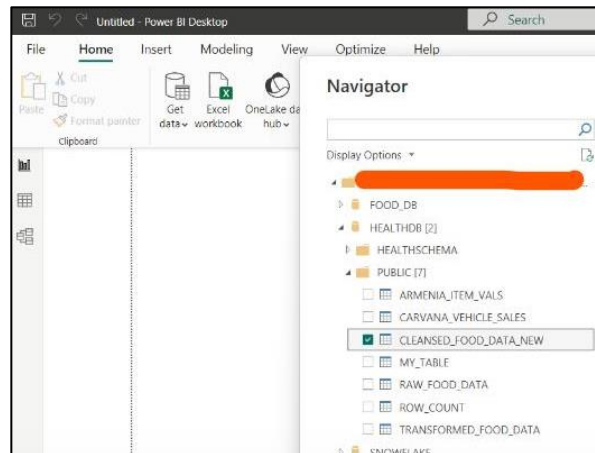
```

15. Create Connection of Snowflake Database with Power BI to create reports and visualizations

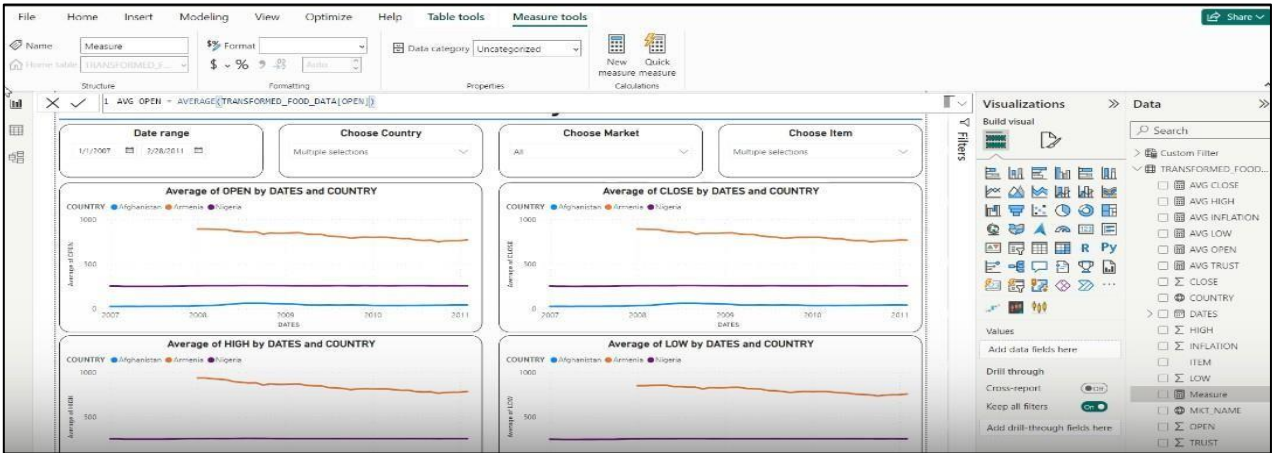
- Open Power BI Desktop and click on "Get Data" in the Home tab.
 - Search for "Snowflake" and select it as the data source.
 - Enter the Snowflake server name, warehouse name, and your Snowflake user credentials (username and password).
- The server name typically follows the format `<account_identifier>.snowflakecomputing.com`. Select the appropriate warehouse for your queries.



- Browse to the relevant schema and select the table that contains the cleaned data
- Data Flow:** Power BI fetches the data from the selected table in Snowflake and loads it into the report for visualizations

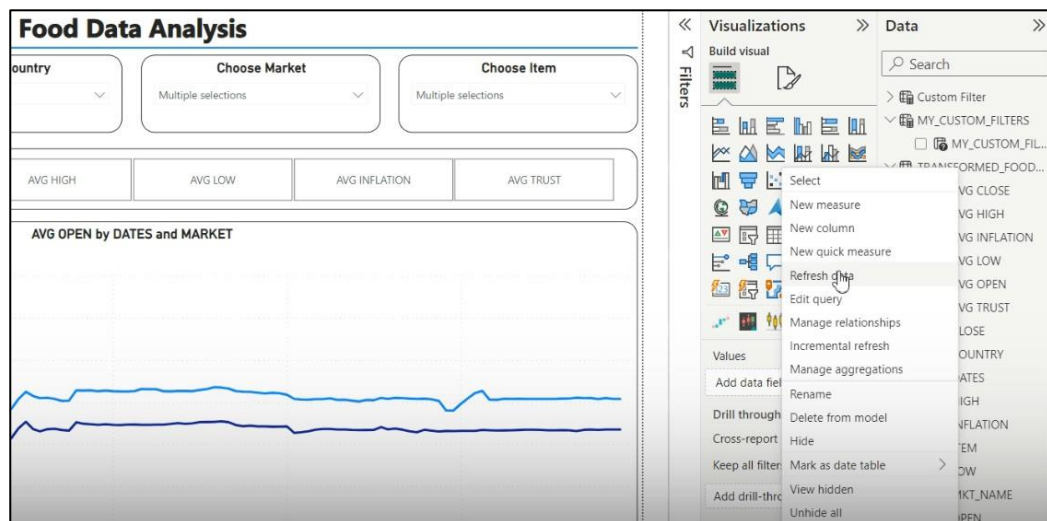


16. Create Visualizations and reports on top of the cleaned data to extract hidden insights



17. Select your transformed table and right click it.

18. Use data refresh option in Power BI to update data whenever there is a change in destination S3 Bucket.



Data Flow: This structured approach ensures a seamless flow from initiating API calls to Snowflake, transforming data, scheduling refreshes in Power BI, and actively monitoring refresh status. By promptly resolving issues encountered during refresh processes, stakeholders can rely on accurate and timely insights for informed decision-making.

Summary

- In this training, we created a real-time streaming environment for situations of transforming data and how appropriate agencies can make strategies about them.
- The reusability of the proposed pipeline in the various industries was discussed.
- Initially, we discussed the concept of Global Food Market Data and how the value and use of data can degrade and change over time, respectively.
- Real Stream Data workflows were defined
- We used Global Food Market data for the use case and defined global industry standard pipelines for the incoming data stream through API Calls.
- The architecture was divided into Raw-Layer, Transformation-Layer, and Real-time Layer to best manage, analyze, and take actions based on the global food market severity.
 - In the Raw Layer, we created an S3 bucket and uploaded our dataset into it. We developed an API using the FastAPI application with AWS App Runner and integrated the S3 bucket with the API to use its objects as endpoints. When we made an API call, data streams were processed and pushed from the Kinesis client to Kinesis Firehose via Kinesis Data Streams.
 - In the Transformation Layer, We implemented a Lambda function to process streaming data from Kinesis Firehose. This function reads the incoming stream logs, transforms the data as specified, and transforms the logs into CSV files.
 - In the Transformation layer, we transformed the data from the staging layer using Snowpark Sessions, filtering raw data with valuable predictability and storing them in a new table different from the staging table.
 - In the Real-Time layer, the workflow is defined as upon making an API call, the data streams were processed and pushed from the Kinesis client to Kinesis Firehose via Kinesis Data Streams. We transformed the data streams into CSV format, partitioned and loaded them into the S3 bucket, and configured Snowpipe to auto-ingest data into a Snowflake staging table using event notifications configuration in the S3 Bucket. We cleaned the raw data using Snowpark sessions, ensuring data integrity and accuracy, and deployed the cleaning logic into stored procedures which get triggered by new S3 data. Then, we created a task using a cron job to schedule stored procedures. Finally, we established a data connection from Power BI to Snowflake, created reports and visualizations, and automated the data loading and visualization updates, with the data refresh functionality which automatically loaded data from the destination S3 bucket.
- Various AWS services were leveraged to implement the concepts with the most efficient cost-optimized configurations that can even serve the amount of demo and data used.